HỌC VIỆN CÔNG NGHỆ BƯU CHÍNH VIỄN THÔNG

Học phần INT4418 Dữ liệu lớn

BÁO CÁO BÀI TẬP LỚN

Premise retrieval cho Lean 4 bằng BM25F và đặc trưng cấu trúc trên mathlib4

Nhóm 3. Lean Premise Retrieval: truy hồi thưa và truy hồi lai

Giảng viên hướng dẫn: TS. Nguyễn Kiều Linh

STT	Họ và tên	Mã học viên
1	Nguyễn Đức Hoàng	B25CHHT096
2	Trần Duy Khánh	B25CHHT100
3	Nguyễn Mạnh Cường	B25CHHT082
4	Vũ Trung Kiên	B25CHHT101
5	Trần Đình Dũng	B25CHHT089
6	Nguyễn Ngọc Minh	B25CHHT106

Tháng 10 năm 2026

Tóm tắt

Báo cáo trình bày một hệ thống premise retrieval cho Lean 4 trên thư viện mathlib4, kết hợp BM25F với các đặc trưng cấu trúc có chi phí tính toán thấp. Dữ liệu là LeanDojo Benchmark 4 bản v10 với 180.957 khai báo; sau khi lọc, tập test của split `random` còn 2.641 truy vấn, mỗi truy vấn là một proof state. Chỉ mục đảo được xây theo mô hình MapReduce trên Apache Spark và bằng một chương trình tuần tự; hai cách cho kết quả trùng từng byte. Ở cấu hình V4, BM25F dùng tokenizer riêng cho Lean trên ba trường name, namespace và statement, bộ lọc chỉ giữ accessible premise (premise dùng được tại vị trí chứng minh theo quan hệ import), rồi hồi quy logistic rerank 100 candidate bằng đặc trưng cấu trúc, popularity và PageRank. Trên tập test `random`, Recall@10 tăng từ 0,100 của BM25 mặc định lên 0,243. V5 bổ sung candidate từ lịch sử sử dụng trong tập train, đạt 0,281 trên tập test và 0,332 so với 0,310 của V4 trên holdout lấy từ phần train chưa dùng để huấn luyện reranker. Tuy vậy, trên holdout chưa có đủ bằng chứng cho thấy V5 tốt hơn đối chứng V4 lấy top 240 candidate, và trên tập test `novel_premises`, Recall@10 trên unseen premise giảm từ 0,444 xuống 0,359. Trong cùng một phiên đo tải, P95 latency ở 10 client của V4 và V5 là 103,2 và 116,7 ms; ở 50 client, thông lượng của V5 giảm từ 132 xuống 116 truy vấn mỗi giây. Dịch vụ bị giới hạn bởi CPU, và Spark local[8] nhanh hơn bản tuần tự 3,84 lần trên corpus nhân bản 3,6 triệu bản ghi. Mọi phép đo thực hiện trên một máy; so sánh với hệ thống dense retrieval của Nhóm 4 chưa hoàn tất.

Từ khóa: premise retrieval, Lean 4, mathlib4, BM25F, chỉ mục đảo, MapReduce, Apache Spark.

Mục lục

Tóm tắt **i**

Danh sách hình vẽ **iv**

Danh sách bảng **iv**

1 Giới thiệu **1**

2 Công trình liên quan **2**

3 Dữ liệu **3**
3.1 Nguồn, phiên bản và lược đồ . . . 3
3.2 Truy vấn và gold premise . . . 3
3.3 Split và tập đánh giá . . . 3
3.4 Kiểm tra chất lượng dữ liệu . . . 4

4 Phương pháp và kiến trúc **5**
4.1 Tổng quan . . . 5
4.2 Tokenizer cho Lean . . . 5
4.3 Xây chỉ mục theo mô hình MapReduce . . . 5
4.4 Hàm xếp hạng BM25 và BM25F . . . 6
4.5 Bộ lọc accessible premise . . . 6
4.6 Reranker cấu trúc . . . 7
4.7 Dịch vụ truy vấn . . . 7

5 Thiết lập thí nghiệm **8**
5.1 Môi trường và các biến thể . . . 8
5.2 Tinh tham số và huấn luyện . . . 8
5.3 Quy trình phát triển và tập holdout . . . 8
5.4 Thiết kế thí nghiệm quy mô và tải . . . 9
5.5 Chỉ số đánh giá và kiểm định . . . 9

6 Kết quả **10**
6.1 Chất lượng truy hồi . . . 10
6.2 Độ bền của kết quả V5 . . . 11
6.3 Đánh đổi giữa chất lượng và độ trễ . . . 12
6.4 Ảnh hưởng của kích thước kho . . . 13
6.5 Độ trễ dưới tải đồng thời . . . 14
6.6 Xây chỉ mục theo lô . . . 14

7 Ablation, phân tích lỗi và hạn chế **16**
7.1 Ablation của reranker . . . 16
7.2 Ablation của đường xây chỉ mục . . . 17

7.3 Phân tích lỗi 17
7.4 Một lỗi được phát hiện nhờ phân tích lỗi 19
7.5 Hạn chế và threats to validity 19

8 Kết luận và hướng phát triển **20**

Tài liệu tham khảo **21**

A Khai báo sử dụng AI và đóng góp **22**
A.1 Khai báo sử dụng AI 22
A.2 Phân công và đóng góp 22

B Tái lập kết quả **23**
B.1 Lệnh tái lập 23
B.2 Ví dụ một bản ghi lần chạy 23

C Bảng bổ sung **25**
C.1 Dữ liệu và môi trường 25
C.2 Truy hồi, phục vụ và xây chỉ mục 26
C.3 Thiết kế thí nghiệm 29

Danh sách hình vẽ

4.1 Kiến trúc hệ thống 5

6.1 Đánh đổi chất lượng và độ trễ; độ trễ dưới tải 12
6.2 Chất lượng và độ trễ theo kích thước chỉ mục 14
6.3 Xây chỉ mục bằng engine Arrow 15

Danh sách bảng

3.1 Luồng lọc dữ liệu theo split 4

4.1 Đặc trưng của reranker 7

5.1 Các biến thể truy hồi và cấu hình đối chứng 8

6.1 Chất lượng truy hồi trên toàn bộ tập test 10
6.2 Chênh lệch Recall@10 giữa các bước cải tiến 10
6.3 Độ bền của kết quả V5 11
6.4 Recall trên unseen premise và seen premise 12
6.5 V4, V4-K240 và V5 dưới tải cùng phiên 13
6.6 Thời gian xây chỉ mục bằng engine Arrow 15

7.1 Ablation của reranker 16
7.2 Đối chứng namespace và đặc trưng phụ thuộc 16
7.3 Phân loại tự động các lỗi của V4 17
7.4 Ví dụ xuyên suốt 18
7.5 Phân rã giới hạn: candidate và xếp hạng 18

A.1 Vai trò, người phụ trách và phần mã tương ứng 22

C.1 Nguồn dữ liệu 25
C.2 Các bảng sau trích xuất 25
C.3 Môi trường thí nghiệm 26
C.4 Bối cảnh: số liệu công bố của LeanDojo 26
C.5 Tinh chỉnh số candidate bổ sung của V5 27
C.6 BM25 của Lucene so với BM25 tự cài đặt 27
C.7 Thời gian theo bước của một truy vấn 27
C.8 Mở rộng dịch vụ theo số worker 28
C.9 So sánh hai engine Spark 28
C.10 Thiết kế thí nghiệm 29

CHƯƠNG 1

Giới thiệu

Lean 4 là một trợ lý chứng minh đồng thời là ngôn ngữ lập trình [1], và mathlib4 là thư viện toán học lớn nhất viết bằng Lean [2]. Khi chứng minh một định lý mới, người dùng hoặc bộ chứng minh tự động phải chọn trong thư viện những định lý, bổ đề và định nghĩa có thể dùng ở bước tiếp theo. Các đối tượng này được gọi là *premise*. Bài toán *premise retrieval* được phát biểu như sau: cho proof state (trạng thái chứng minh) hiện tại, trả về một danh sách ngắn các premise có khả năng được dùng trong tactic kế tiếp [3]. Chất lượng của danh sách này ảnh hưởng trực tiếp tới khả năng thành công của bộ chứng minh, vì bộ chứng minh không thể xét toàn bộ thư viện ở mỗi bước. Có hai hướng tiếp cận chính: sparse retrieval so khớp từ khóa trên chỉ mục đảo, điển hình là BM25; dense retrieval mã hóa truy vấn và premise thành vector bằng mô hình neural rồi tìm láng giềng gần nhất; hybrid retrieval kết hợp cả hai. Báo cáo này thuộc hướng sparse retrieval, có bổ sung đặc trưng cấu trúc.

Bài toán có ba đặc điểm của dữ liệu lớn. Thứ nhất là khối lượng: snapshot mathlib4 được dùng có 180.957 khai báo, và mỗi bước chứng minh sinh ra một truy vấn, nên nếu so khớp từng truy vấn với mọi khai báo thì chi phí tăng tuyến tính theo kích thước kho và theo số truy vấn cần phục vụ. Thứ hai là phân bố lệch: từ khóa `theorem` xuất hiện trong 109.826 phát biểu, ký hiệu = trong hơn 80.000 phát biểu. Các token này tạo ra khóa nóng (hot key) ở pha shuffle khi xây chỉ mục và posting list dài khi truy vấn. Thứ ba là yêu cầu phục vụ: hệ thống phải trả lời nhiều truy vấn đồng thời và giữ P95 latency ở mức thấp, đồng thời chỉ mục phải xây lại được khi thư viện đổi phiên bản.

Nhóm đặt hai câu hỏi nghiên cứu theo đề bài:

RQ1. BM25 có đủ mạnh cho dữ liệu Lean, hay cần bổ sung trường ký hiệu, namespace và thông tin phụ thuộc?

RQ2. Kết hợp điểm BM25F với các đặc trưng cấu trúc có làm tăng Recall@k mà vẫn giữ được P95 latency thấp hay không?

Hai câu hỏi cùng hướng tới câu hỏi trung tâm của đề: đặc trưng cấu trúc thực sự cải thiện recall hay chỉ làm tăng chi phí. Nhóm chọn hai chỉ số chính là Recall@10 cho chất lượng và P95 latency ở 10 client đồng thời cho chi phí; các chỉ số khác dùng để giải thích đánh đổi.

Đóng góp của nhóm gồm bốn phần:

1. Một đặc tả benchmark và pipeline trích xuất tái lập được từ LeanDojo Benchmark 4, có data contract, bảng dead-letter cho mọi bản ghi bị loại, thống kê luồng lọc theo từng tập và kiểm tra tự động 20 truy vấn mẫu.
2. Một chỉ mục đảo xây theo mô hình MapReduce bằng hai engine Spark, trùng từng byte với bản tuần tự, cùng thí nghiệm xây chỉ mục tới 3,6 triệu bản ghi (corpus nhân bản) trên Windows và Linux của cùng một máy, có đo shuffle, bộ nhớ, độ lệch và nén.
3. Một hệ thống truy hồi hai giai đoạn gồm BM25F và reranker cấu trúc (V4), cùng biến thể V5 bổ sung candidate từ lịch sử sử dụng trong tập train, được đối chiếu với V4 lấy top 240 candidate và kiểm tra trên một holdout chưa dùng để huấn luyện hay chọn cấu hình.
4. Một bộ đánh giá gồm chất lượng và độ trễ trên bốn kích thước chỉ mục và ba mức tải, khoảng tin cậy bootstrap theo cụm theorem và kiểm định hoán vị theo cụm có hiệu chỉnh Holm, ablation, phân rã giới hạn giữa bước tạo candidate và bước xếp hạng, và phân tích lỗi.

Phần còn lại của báo cáo được tổ chức như sau. Chương 2 điểm qua công trình liên quan. Chương 3 mô tả dữ liệu và định nghĩa benchmark. Chương 4 trình bày kiến trúc và thuật toán. Chương 5 nêu thiết lập thí nghiệm, Chương 6 trình bày kết quả, Chương 7 trình bày ablation, phân tích lỗi, hạn chế và threats to validity. Chương 8 kết luận.

CHƯƠNG 2

Công trình liên quan

Sparse retrieval. BM25 và mở rộng nhiều trường BM25F được Robertson và Zaragoza tổng kết trong khung xác suất về độ liên quan [5]. BM25F chuẩn hóa tần suất của từng trường theo độ dài trường rồi mới áp dụng hàm bão hòa, nên một term xuất hiện ở nhiều trường không bị tính quá mức như khi cộng điểm BM25 của từng trường. Cấu trúc chỉ mục đảo (inverted index), mã hóa khoảng cách (d-gap), mã Variable-byte và mã gamma được trình bày trong giáo trình của Manning và cộng sự [6]; Scholer và cộng sự chỉ ra rằng mã theo byte giải mã nhanh hơn đáng kể so với mã theo bit với tỉ lệ nén chấp nhận được [7]. Hệ thống của nhóm dùng BM25F làm giai đoạn đầu và đo lại đánh đổi nén trên dữ liệu Lean.

Chọn premise trong trợ lý chứng minh. MePo, bộ lọc liên quan dùng trong Sledgehammer của Isabelle, chọn premise theo mức trùng ký hiệu với mục tiêu và mở rộng dần theo các ký hiệu mới [8]. DeepMath dùng mô hình chuỗi học sâu để chọn premise cho thư viện Mizar [9]. Magnushammer học biểu diễn bằng transformer theo phương pháp tương phản và vượt Sledgehammer trên Isabelle [10]. LeanDojo xây công cụ trích xuất dữ liệu từ Lean, công bố LeanDojo Benchmark với hai split `random` và `novel_premises`, và đề xuất ReProver, một dense retriever chỉ tìm trong các accessible premise, tức premise dùng được tại vị trí đang chứng minh [3, 4]. Nhóm dùng lại benchmark và ý tưởng accessible premise, nhưng giữ toàn bộ hệ thống ở dạng sparse retrieval và không dùng mô hình neural; hướng dense retrieval do Nhóm 4 của lớp khảo sát.

Xây chỉ mục phân tán. MapReduce [11] và Spark với mô hình RDD [12, 13] là hai mô hình tính toán phổ biến để xây chỉ mục đảo theo lô. Lin và Dyer mô tả các mẫu thiết kế cho bài toán này, trong đó có kết hợp cục bộ trong mapper và chuyển giá trị thành một phần của khóa để reducer nhận dữ liệu đã sắp xếp [14]. Hệ thống của nhóm cài đặt các mẫu này trên Spark và đo tác động của chúng.

Tín hiệu đồ thị và trộn xếp hạng. PageRank [15] được dùng trên đồ thị theorem tới premise để ước lượng mức quan trọng của premise. Reciprocal Rank Fusion [16] trộn hai danh sách xếp hạng mà không cần chuẩn hóa điểm, phù hợp để kết hợp hệ thống sparse retrieval của Nhóm 3 với hệ thống dense retrieval của Nhóm 4 thành hybrid retrieval.

Khoảng trống. Các hệ thống premise retrieval gần đây chủ yếu dùng dense retrieval và báo cáo chất lượng trên một cấu hình phần cứng cố định. Ít công trình đo riêng xem BM25F cộng các đặc trưng cấu trúc rẻ đạt tới mức nào, và chi phí xây chỉ mục cùng độ trễ thay đổi ra sao khi kho tăng. Báo cáo này tập trung vào khoảng trống đó.

CHƯƠNG 3

Dữ liệu

3.1 Nguồn, phiên bản và lược đồ

Dữ liệu là LeanDojo Benchmark 4 bản v10 (Bảng C.1 ở Phụ lục C), tải từ Zenodo [17] và kiểm tra md5 trước khi giải nén; script ghi manifest gồm đường dẫn, kích thước và mã băm. Dữ liệu đầy đủ không được đưa vào Git; kho mã chỉ chứa mẫu gồm 1.000 premise và 200 truy vấn cùng script tải và tạo lại dữ liệu. Pipeline trích xuất chuyển JSON sang năm bảng Parquet (Bảng C.2). Mỗi premise được định danh bằng cặp (`full_name`, `path`) vì có 57 tên xuất hiện đồng thời trong Lean core và Mathlib, ví dụ `not_le`; mã `premise_id` là thứ tự của cặp này nên ổn định với cùng snapshot. Trường `statement` là mã nguồn cắt tại dấu `:=` ở cấp ngoài cùng, tại `where` hoặc tại nhánh `|` đầu dòng, để không đưa phần chứng minh vào chỉ mục; mô tả từng cột nằm trong `docs/data_dictionary.md`. Corpus gồm 156.947 khai báo của Mathlib, 19.619 của Lean core, 2.883 của Batteries và phần nhỏ của các thư viện khác; theo loại có 109.823 `theorem`, 36.826 `def`, 16.969 `lemma` và 10.071 `instance`.

3.2 Truy vấn và gold premise

Một truy vấn là trường `state_before` của một tactic có ít nhất một premise được chú thích; tập gold premise là các premise trong chú thích đó. Văn bản của tactic không được đưa vào truy vấn vì nó chứa sẵn tên premise cần tìm. Ví dụ, trong truy vấn `random:val:36:0`, mục tiêu là chứng minh tích của `a`, nghịch đảo của `b` và `b` bằng `a` trong một monoid; tactic `simp [mul_assoc]` dùng premise `mul_assoc`, còn hệ thống chỉ được thấy proof state. Truy vấn này là truy vấn đầu tiên trong mẫu kiểm tra ở Mục 3.4 và được dùng làm ví dụ xuyên suốt (Bảng 7.4). Ngoài proof state, hệ thống được dùng tên theorem đang chứng minh, tệp chứa nó và dòng bắt đầu của nó; bộ lọc accessible premise luôn loại chính theorem này ra.

Có 115.736 lượt gold premise không có trong corpus và bị loại. Hiện tượng này đến từ bản thân benchmark: các bổ đề sinh tự động bởi `@[to_additive]` như `add_comm`, các constructor như `Or.inl` và các trường của structure như `Iff.mp` không phải khai báo tường minh trong `corpus.jsonl`. Sau khi loại, 20.544 truy vấn không còn gold premise nào và cũng bị loại; mọi bản ghi bị loại được ghi vào bảng `dead_letter` kèm lý do. Vì vậy recall báo cáo chỉ tính premise có trong corpus và cao hơn recall mà bộ chứng minh gặp thực tế.

3.3 Split và tập đánh giá

Nhóm giữ nguyên hai split của LeanDojo. Split `random` là split chính; split `novel_premises` được thiết kế để khó hơn: mỗi chứng minh trong tập test dùng ít nhất một premise chưa từng được dùng trong tập train. Mọi tham số chỉ được chọn trên validation. Bảng 3.1 theo dõi số lượng qua từng bước lọc. Mỗi split có 2.000 theorem trong tập test, nhưng chỉ 882 theorem của `random` và 1.196 theorem của `novel_premises` còn ít nhất một truy vấn, tức 44,1% và 59,8%. Nguyên nhân chính là khoảng một nửa số theorem được chứng minh trực tiếp bằng term nên không có tactic nào. Kết quả chất lượng vì vậy đại diện cho các chứng minh dạng tactic có chú thích premise, không phải cho mọi theorem. Con số 315.014 của bảng `queries` là tổng lượt truy vấn của cả hai split; trong từng tập, số proof state khác nhau ít hơn số truy vấn chưa tới 1%.

Tính chất novel sau khi lọc. Bảo đảm của split `novel_premises` đặt ở cấp chứng minh, còn đánh giá diễn ra ở cấp tactic sau khi lọc nhãn. Gọi một gold premise là unseen premise nếu nó không xuất hiện trong nhãn của tập train cùng split (khác với premise vắng khỏi corpus), ngược lại là seen premise. Chỉ 2.506 trên 9.735 cặp (truy vấn, gold premise) của tập test, tức 25,7%, là unseen premise, gồm 1.933 premise khác nhau; 103 trên 1.196 theorem được chấm không còn unseen premise nào. Recall tổng thể của split này vì vậy chủ yếu là recall trên seen premise; Mục 6.2 báo cáo riêng recall trên unseen premise.

Bảng 3.1. Luồng lọc dữ liệu theo split và tập

	`random`			`novel_premises`		
Bước	train	val	test	train	val	test
Theorem gốc	118.517	2.000	2.000	118.517	2.000	2.000
có ít nhất một tactic	59.557	995	992	59.050	1.245	1.249
có tactic được chú thích premise	53.787	892	898	53.154	1.207	1.216
còn ít nhất một truy vấn	52.663	880	882	52.047	1.182	1.196
tỉ lệ so với theorem gốc	44,4%	44,0%	44,1%	43,9%	59,1%	59,8%
Tactic	250.814	4.260	4.506	246.714	6.240	6.626
được chú thích premise	162.262	2.681	2.836	159.234	4.165	4.380
bị loại vì mọi gold premise nằm ngoài corpus	9.932	145	195	9.775	258	239
Truy vấn giữ lại	152.330	2.536	2.641	149.459	3.907	4.141
proof state khác nhau	151.134	2.530	2.623	148.288	3.876	4.109
Lượt gold premise nằm ngoài corpus	55.971	867	1.030	54.047	1.886	1.935

Ghi chú: phần lớn theorem bị loại vì được chứng minh không qua tactic (chứng minh dạng term) nên không sinh truy vấn nào.

Tập train được chia đôi cố định theo md5 của tên theorem: train A dùng để tính thống kê phụ thuộc như popularity và PageRank, train B dùng để huấn luyện reranker, để đặc trưng không chứa nhãn của chính truy vấn dùng để huấn luyện. Thống kê và mô hình cho `novel_premises` được tính lại từ tập train của split này, vì tập train của `random` chứa các theorem thuộc tập test của `novel_premises`.

Tập con cho thí nghiệm quy mô. Nhóm lấy mẫu 500 truy vấn trong tập test `random` (seed 42) và tạo bốn chỉ mục lồng nhau 10K, 50K, 100K và 181K premise, đều chứa 870 gold premise của mẫu, phần còn lại lấy ngẫu nhiên. Thí nghiệm xây chỉ mục dùng thêm hai corpus nhân bản nội dung gấp 5 lần (904.785 bản ghi) và gấp 20 lần (3.619.140 bản ghi), chỉ để đo thời gian, bộ nhớ và độ lệch.

3.4 Kiểm tra chất lượng dữ liệu

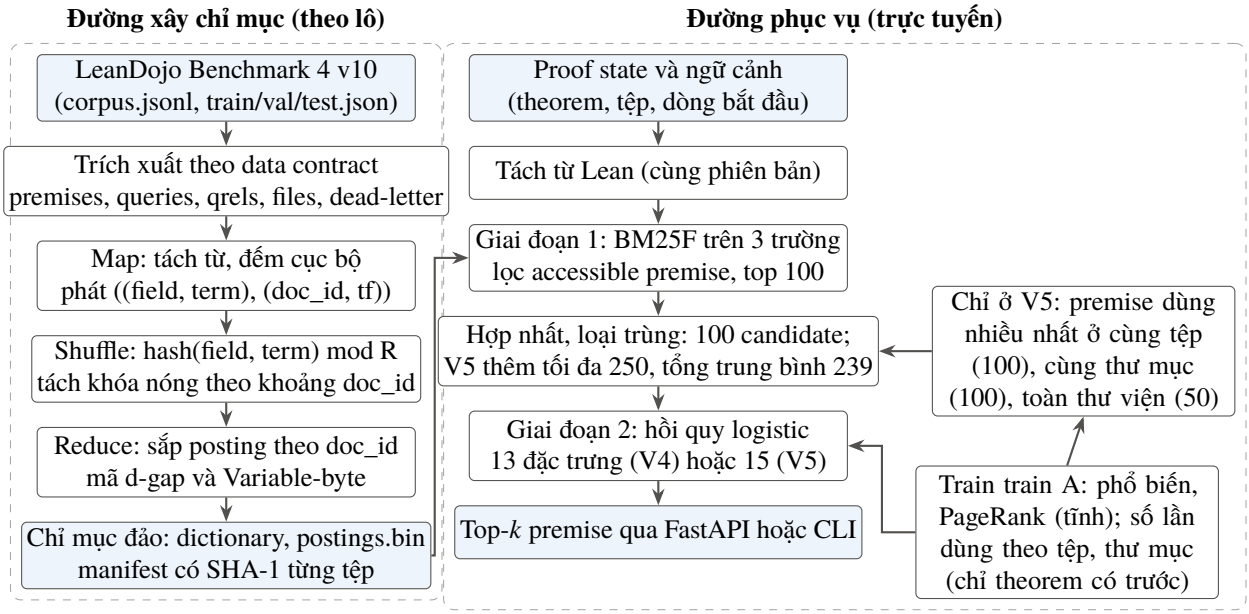
Script `make_check_queries.py` lấy 20 truy vấn validation (seed 42, mỗi lĩnh vực nhiều nhất hai truy vấn) và kiểm tra tự động bốn điều kiện so với tactic gốc: gold premise được đánh dấu trong tactic, là accessible premise của theorem, không phải chính theorem, và mọi premise bị loại đều có trong dead-letter. Cả 20 truy vấn đạt bốn điều kiện; 16 bản ghi trùng hoàn toàn đã được loại. Bước đối chiếu thủ công từng truy vấn và ký xác nhận trong `docs/check_queries.md` chưa được thực hiện. [Người kiểm tra đối chiếu 20 truy vấn và ký xác nhận.]

CHƯƠNG 4

Phương pháp và kiến trúc

4.1 Tổng quan

Hệ thống gồm hai đường xử lý (Hình 4.1). Đường xây chỉ mục chạy theo lô: trích xuất dữ liệu, tokenize và xây chỉ mục đảo theo mô hình MapReduce. Đường phục vụ chạy trực tuyến: nhận proof state cùng ngữ cảnh, tokenize bằng đúng tokenizer và phiên bản đã dùng khi xây chỉ mục, dùng BM25F trên ba trường name, namespace và statement để lấy top 100 accessible premise làm candidate, rồi reranker dùng hồi quy logistic để xếp hạng lại dựa trên các đặc trưng cấu trúc, popularity và PageRank (V4). V5 bổ sung candidate từ lịch sử sử dụng trong train A và thêm hai đặc trưng `file_usage`, `dir_usage`.



Hình 4.1. Kiến trúc hệ thống. Bản xây chỉ mục tuần tự và bản Spark cho cùng một chỉ mục. Candidate bổ sung của V5 cũng qua bộ lọc accessible premise; 239 là tổng số candidate trung bình của V5 trên tập test `random`, gồm cả 100 candidate của BM25F.

4.2 Tokenizer cho Lean

Tokenizer mặc định của baseline chỉ lấy các chuỗi chữ và số rồi chuyển về chữ thường, nên làm mất ký hiệu toán học và tên ghép. Tokenizer Lean giữ tên đầy đủ có dấu chấm đồng thời tách thành các thành phần, ví dụ `Nat.add_comm` sinh ra `nat.add_comm`, `nat`, `add_comm`, `add` và `comm`; tách thêm theo camelCase (`natAbs` thành `nat` và `abs`); giữ các ký hiệu như $\forall$, $\to$, $\leq$, $\neq$, $\mathbb{N}$, $\mathbb{R}$ làm token; và bỏ biến một chữ cái ASCII. Tokenizer không dùng chuẩn hóa NFKC vì NFKC biến $\mathbb{R}$ thành chữ R thường. Phiên bản tokenizer được ghi trong manifest của chỉ mục để đường phục vụ dùng đúng phiên bản.

4.3 Xây chỉ mục theo mô hình MapReduce

Đơn vị dữ liệu là một premise với ba trường `name`, `namespace` và `statement`. Nhóm cài đặt hai engine Spark cho cùng một đầu ra; engine thứ hai được viết lại sau khi đo engine thứ nhất (Mục 7.2).

Engine RDD. Driver đọc dữ liệu rồi phân phối cho các executor. Mapper phân tích văn bản, đếm tần suất của term trong tài liệu ngay tại chỗ (kết hợp cục bộ) rồi phát cặp ((field, term), (doc_id, tf)). Pha shuffle chia dữ liệu theo $\text{hash}(\text{field}, \text{term}) \bmod R$; khóa nóng có tần suất tài liệu vượt 20.000 được tách thành S phần theo khoảng $\lfloor \text{doc_id} \cdot S/N \rfloor$, nên các phần nối lại theo thứ tự mà không phải sắp xếp lại. Reducer sắp posting theo doc_id, tính tần suất, mã hóa và ghi tệp con; driver trộn từ điển theo thứ tự (field, term) cố định và ghi manifest chứa tham số, thống kê và mã SHA-1 của từng tệp. Nhờ thứ tự cố định, bản Spark và bản tuần tự cho chỉ mục trùng nhau từng byte, được kiểm tra bằng cách so mã băm.

Engine Arrow. Engine này giữ nguyên đầu ra nhưng thay đổi ba điểm. Thứ nhất, executor tự đọc Parquet qua Spark SQL; driver chỉ đọc cột `premise_id` để đánh doc_id, nên bộ nhớ của driver không tăng theo nội dung corpus. Thứ hai, mã Python nhận dữ liệu theo lô Arrow (`mapInArrow`) và gộp posting ở mức partition: mỗi partition phát một bản ghi cho mỗi cặp (field, term) nó chứa, mang cả dãy doc_id và dãy tf dưới dạng mảng nhị phân. Đây là mẫu kết hợp trong mapper của Lin và Dyer [14] áp dụng cho cả partition. Thứ ba, shuffle và sắp xếp chạy trong JVM, tần suất tài liệu và độ dài trường đi kèm dưới dạng khóa đặc biệt, nên chỉ còn một lượt shuffle dữ liệu chính thay vì hai.

Mỗi posting list lưu dãy khoảng cách doc_id (d-gap) và dãy tf, mã bằng số nguyên 32 bit, Variable-byte (VB, mặc định) hoặc gamma; ví dụ của giáo trình [6] (dãy 3, 7, 1, 119 mã VB thành `83 87 81 F7`, mã gamma thành 22 bit) nằm trong unit test. Gọi T là tổng số token, P là số posting, K là số cặp (field, term) và R là số partition. Pha map tốn $O(T)$. Kết hợp cục bộ của engine RDD giảm số bản ghi shuffle từ T xuống P; với engine Arrow, số bản ghi shuffle không vượt quá $\min(P, K \cdot R)$, còn tổng dung lượng vẫn là $O(P)$. Khi corpus lớn lên, P tăng theo số tài liệu còn K tăng chậm hơn; với corpus nhân bản ở Chương 6, K không đổi nên đây là trường hợp thuận lợi nhất cho engine Arrow. Pha reduce sắp xếp tổng cộng không quá $O(P \log P_{\max})$. Với chỉ mục ba trường của corpus đầy đủ, $P = 4.323.308$ và $K = 481.002$.

4.4 Hàm xếp hạng BM25 và BM25F

Với truy vấn q và tài liệu d, BM25F tính tần suất đã chuẩn hóa của term t và điểm của tài liệu là

$$\tilde{tf}(t,d) = \sum_f w_f \cdot \frac{tf_f(t,d)}{1 - b_f + b_f \cdot \ell_f(d)/\bar{\ell}_f}, \qquad s(q,d) = \sum_{t \in q} \text{qtf}(t) \cdot \text{idf}(t) \cdot \frac{\tilde{tf}(t,d)\,(k_1+1)}{\tilde{tf}(t,d) + k_1}, \tag{4.1}$$

trong đó w_f là trọng số trường f, b_f là hệ số chuẩn hóa độ dài, $\ell_f(d)$ là độ dài trường, $\bar{\ell}_f$ là độ dài trung bình và $\text{idf}(t) = \ln\left(1 + (N - \text{df}(t) + 0{,}5)/(\text{df}(t) + 0{,}5)\right)$ không âm như trong Lucene. Khi chỉ có một trường và $w = 1$, công thức trở về BM25. Hệ số $\text{qtf}(t)$ là số lần term xuất hiện trong truy vấn hoặc bằng 1 (chế độ nhị phân, phù hợp với proof state nơi tên giả thiết lặp lại nhiều lần). Chi phí của một truy vấn phụ thuộc vào tổng độ dài các posting list được đọc, $\sum_{t \in q} \sum_f \text{df}_f(t)$, cộng với thao tác gộp điểm. Cách làm này tránh tính điểm riêng cho từng tài liệu nhưng không độc lập với kích thước kho: posting list của các token phổ biến như `=` hay $\to$ dài ra tuyến tính theo số tài liệu N. Cài đặt hiện tại gộp điểm bằng `np.unique`, nên có thêm chi phí sắp xếp $O(M \log M)$ với M là số posting được đọc, và bộ lọc accessible premise tạo mặt nạ trên toàn bộ N premise.

4.5 Bộ lọc accessible premise

Một premise là accessible premise của theorem đang chứng minh nếu nó nằm trong tệp được import trực tiếp hoặc gián tiếp bởi tệp chứa theorem, hoặc nằm trong cùng tệp và đã khai báo xong trước theorem; chỉ những premise này mới dùng được tại vị trí chứng minh. Hệ thống tính bao đóng import trên 5.674 tệp, lưu đệm theo tệp rồi tạo mặt nạ trên toàn bộ premise. Với premise cùng tệp, điều kiện là dòng *kết thúc* của premise nhỏ hơn dòng bắt đầu của theorem (Mục 7.4 giải thích lý do). Bộ lọc được áp dụng ở giai đoạn 1, trước khi chọn 100 candidate.

4.6 Reranker cấu trúc

Giai đoạn 2 rerank 100 candidate bằng hồi quy logistic trên 13 đặc trưng ở Bảng 4.1. Đặc trưng chỉ dùng proof state, ngữ cảnh của truy vấn và thống kê tính trên train A, không dùng nhãn của validation hay tập test. Mô hình được huấn luyện trên 6.000 truy vấn lấy ngẫu nhiên từ train B, gồm 600.000 cặp (truy vấn, candidate) với 4.492 cặp dương; candidate lúc huấn luyện được sinh bằng đúng giai đoạn 1 và bộ lọc như lúc chạy. Đặc trưng được chuẩn hóa theo tập train; hệ số điều chuẩn $C = 1$. Đồ thị phụ thuộc có cạnh từ mỗi theorem của train A tới các premise nó dùng, gồm 129.983 cạnh và 41.703 premise; PageRank dùng hệ số damping 0,85 và hội tụ sau 11 vòng lặp.

Bảng 4.1. Đặc trưng của reranker. V3 dùng 11 đặc trưng đầu, V4 dùng 13, V5 dùng đủ 15

Nhóm	Đặc trưng	Ý nghĩa
Điểm giai đoạn 1	`bm25_rel`, `bm25_rr`	điểm BM25F chia cho điểm cao nhất của truy vấn; nghịch đảo hạng
Namespace	`ns_in_query`, `ns_thm`	tỉ lệ thành phần namespace của premise có trong truy vấn; độ dài tiền tố namespace chung với theorem
Vị trí	`same_file`, `dir_overlap`	premise cùng tệp với theorem; tỉ lệ thư mục chung
Ký hiệu	`sym_jaccard`	hệ số Jaccard giữa tập ký hiệu toán của mục tiêu và của phát biểu
Loại khai báo	`kind_thm`, `kind_def`, `kind_inst`, `kind_type`	theorem hoặc lemma; định nghĩa; instance; structure, class hoặc inductive
Phụ thuộc	`popularity`, `pagerank`	logarit của số theorem train A dùng premise (cộng 1), đã chuẩn hóa; logarit PageRank đưa về khoảng từ 0 đến 1
Sử dụng (V5)	`file_usage`, `dir_usage`	logarit của số theorem train A có trước, cùng tệp hoặc cùng thư mục được import, dùng premise; chuẩn hóa theo giá trị lớn nhất

V5: thêm candidate từ lịch sử sử dụng. Phân tích lỗi của V4 trên tập test `random` (Mục 7.3) cho thấy phần lớn gold premise bị bỏ lỡ không lọt vào 100 candidate của BM25F, và nhiều trong số đó là bổ đề tổng quát được dùng ở rất nhiều nơi. V5 bổ sung candidate từ ba nguồn không dựa vào từ khóa: các premise được dùng nhiều nhất bởi những theorem cùng tệp, bởi những theorem cùng thư mục, và các premise phổ biến nhất toàn thư viện; mọi candidate bổ sung đều qua bộ lọc accessible premise. Hai đặc trưng `file_usage` và `dir_usage` đưa số lần sử dụng vào reranker; số candidate của từng nguồn được chọn trên validation (Mục 5.2). Thống kê theo tệp chỉ đếm các theorem train A khai báo trước theorem đang chứng minh, thống kê theo thư mục chỉ đếm thêm theorem ở những tệp được import; ràng buộc này có unit test. Ngược lại, popularity và PageRank là prior tĩnh tính một lần trên toàn bộ train A. Như vậy hệ thống không dùng nhãn của validation hay tập test và chỉ trả về accessible premise, nhưng không mô phỏng đầy đủ tình huống viết một định lý mới tại một thời điểm.

4.7 Dịch vụ truy vấn

Dịch vụ viết bằng FastAPI và chạy 4 tiến trình worker của uvicorn. Mỗi worker nạp chỉ mục bằng ánh xạ bộ nhớ, giữ bộ đệm LRU cho 4.096 posting list đã giải mã và chạy một truy vấn khởi động trước khi nhận yêu cầu. Dịch vụ có các endpoint `/search`, `/batch_search` và `/health`, kèm một CLI cho kiểm tra thủ công và demo.

CHƯƠNG 5

Thiết lập thí nghiệm

5.1 Môi trường và các biến thể

Mọi thí nghiệm chạy trên một máy 6 nhân, 12 luồng, 32 GB RAM (Bảng C.3, Phụ lục C); phiên bản các gói Python được khóa trong `requirements/`. Mỗi lần chạy ghi một bản ghi theo Phụ lục B của hướng dẫn gồm mã lần chạy, thời điểm, phiên bản mã và dữ liệu, cấu hình máy, phần mềm, tham số, seed, kết quả và trạng thái (Mục B.2). Bảng 5.1 liệt kê tám biến thể của chuỗi cải tiến và bốn cấu hình đối chứng. B0 là baseline bắt buộc của đề; B0t là B0 đã tinh tham số với cùng ngân sách như V1. Mỗi cấu hình đối chứng chỉ khác biến thể gốc đúng một điểm và được huấn luyện lại trên cùng dữ liệu.

Bảng 5.1. Các biến thể truy hồi và cấu hình đối chứng

Mã	Mô tả
B0	Baseline: BM25, $k_1 = 1{,}2$, $b = 0{,}75$, qtf đếm, trường statement, tokenizer mặc định
B0t	B0 với $k_1 = 2{,}0$, $b = 1{,}0$, qtf nhị phân, chọn trên validation
V1	B0t với tokenizer Lean; $k_1 = 3{,}0$, $b = 1{,}0$, qtf nhị phân
V2	V1 với BM25F trên ba trường name, namespace, statement; trọng số 1,0; 0,5; 1,0 và $b_f = 1{,}0$
V2a	V2 với bộ lọc accessible premise
V3	V2 với reranker 11 đặc trưng, không lọc, không dùng thống kê phụ thuộc
V4	V3 với hai đặc trưng phụ thuộc (popularity, PageRank) và bộ lọc accessible premise
V5	V4 thêm tối đa 250 candidate từ ba nguồn sử dụng (100 theo tệp, 100 theo thư mục, 50 phổ biến) và hai đặc trưng sử dụng
V4-K240	V4 lấy top 240 của BM25F thay vì 100, xấp xỉ số candidate trung bình của V5 trên `random` và holdout (V5 có 208 trên `novel_premises`); vẫn dùng 13 đặc trưng của V4
V2-nons	V2 với trọng số namespace bằng 0, tức BM25F trên name và statement
V4-poponly	V4 chỉ giữ popularity trong hai đặc trưng phụ thuộc
V4-pronly	V4 chỉ giữ PageRank trong hai đặc trưng phụ thuộc

5.2 Tinh tham số và huấn luyện

Tham số của giai đoạn 1 được chọn bằng tìm kiếm lưới trên validation của `random`: 40 điểm cho B0t và V1, mở rộng thêm 8 điểm vì điểm tốt nhất nằm ở biên lưới, và 20 tổ hợp trọng số trường cho V2. Điểm cao nhất của lưới BM25F (Recall@10 bằng 0,1727) đặt trọng số namespace bằng 0; cấu hình được chọn thấp hơn 0,0005 vì có Recall@5 và Recall@50 cao hơn và dùng đủ ba trường. Quy tắc này không được định trước nên không là bằng chứng cho thấy namespace cần thiết; đối chứng V2-nons kiểm tra trực tiếp câu hỏi đó. Với V5, nhóm thử 8 tổ hợp số candidate bổ sung trên validation (Bảng C.5, Phụ lục C) và chọn tổ hợp 100, 100, 50 có Recall@10 cao nhất. Tham số của B0t đến V4 được chốt trước khi chạy tập test; số candidate của V5 được chọn trên validation, nhưng thiết kế của V5 đã chịu ảnh hưởng của tập test.

5.3 Quy trình phát triển và tập holdout

Tập test ảnh hưởng tới thiết kế hai lần. Lần thứ nhất là lỗi của bộ lọc accessible premise, phát hiện khi phân tích lỗi của V4 trên tập test (Mục 7.4); đây là sửa lỗi tính đúng, không phải chọn tham số. Lần thứ hai là V5:

ý tưởng thêm candidate từ lịch sử sử dụng đến từ phân tích lỗi của V4 trên tập test `random`. Kết quả của V5 trên tập test vì vậy có được sau khi đã xem lỗi trên chính tập đó, chứ không phải kết quả của một cấu hình chốt từ trước.

Sau khi chốt V5, nhóm tạo tập holdout bằng `scripts/make_holdout.py`: lấy theorem từ train B của `random`, loại mọi theorem có truy vấn trong mẫu 6.000 truy vấn huấn luyện reranker, rồi lấy ngẫu nhiên (seed 42) 2.500 truy vấn của 1.138 theorem, không trùng theorem với train A, mẫu huấn luyện, validation hay tập test. Holdout là phần train B chưa dùng để huấn luyện reranker, giữ riêng để xác nhận nội bộ, cùng snapshot và phân phối, không phải benchmark bên ngoài. Vì các theorem có truy vấn trong mẫu huấn luyện bị loại, chứng minh nhiều bước dễ bị loại hơn: holdout có 2,20 truy vấn mỗi theorem so với 2,99 ở tập test `random`. Kết quả trên holdout vì vậy xác nhận trên dữ liệu chưa dùng để huấn luyện hay chọn cấu hình, không chứng minh khả năng tổng quát hóa sang phiên bản thư viện khác. Thứ tự thời gian của các lần chạy, lấy từ run record, nằm trong `docs/process_log.md`: mỗi hệ thống được đánh giá trên holdout một lần và không cấu hình nào được chọn theo holdout. Từ nay holdout không còn là tập chưa xem cho các cải tiến tiếp theo.

5.4 Thiết kế thí nghiệm quy mô và tải

Nhóm dùng bốn kích thước chỉ mục cho chất lượng và độ trễ, sáu kích thước corpus từ 10K đến 3,6 triệu bản ghi cho thời gian xây chỉ mục trên hai nền tảng, và ba mức tải 1, 10 và 50 client (Bảng C.10, Phụ lục C). Thí nghiệm tải chính gồm 36 cấu hình (3 biến thể, 4 kích thước chỉ mục, 3 mức tải), mỗi cấu hình đo 3 lần, mỗi lần 400 yêu cầu sau 50 yêu cầu khởi động, tổng cộng 108 lần đo và 43.200 yêu cầu. Client đa luồng giữ kết nối HTTP liên tục và đo độ trễ đầu cuối của từng yêu cầu.

5.5 Chỉ số đánh giá và kiểm định

Với truy vấn q, tập gold premise G_q và danh sách trả về R_q, Recall@k bằng $|G_q \cap R_q^{(k)}|/|G_q|$ với $R_q^{(k)}$ là k phần tử đầu; MRR là trung bình nghịch đảo hạng của gold premise đầu tiên trong top 100. Chỉ số chính là query-macro, tức trung bình theo truy vấn, mỗi truy vấn có trọng số bằng nhau. Vì có theorem chứa tới 53 truy vấn, nhóm báo thêm theorem-macro: lấy trung bình trong từng theorem rồi trung bình giữa các theorem, mỗi theorem có trọng số bằng nhau. Recall trên unseen premise chỉ tính các unseen premise và chỉ lấy trung bình trên các truy vấn có ít nhất một unseen premise. Với candidate pool C_q trước khi rerank, candidate recall là $|G_q \cap C_q|/|G_q|$, Oracle@k là $\min(k, |G_q \cap C_q|)/|G_q|$ (Recall@k của một reranker hoàn hảo), candidate loss là $1-$ candidate recall và ranking gap là Oracle@10 trừ Recall@10. Chỉ số chi phí gồm P50, P95 latency đầu cuối, QPS, thời gian xây chỉ mục, bộ nhớ đỉnh, dung lượng shuffle và kích thước chỉ mục.

Khoảng tin cậy 95% của chênh lệch giữa hai hệ thống lấy từ paired bootstrap theo cụm [19]: lấy mẫu có hoàn lại 10.000 lần trên tập theorem, mỗi theorem mang theo mọi truy vấn của nó. Giá trị p lấy từ kiểm định hoán vị ghép cặp theo cụm [20] với giả thuyết không là trong mỗi theorem, kết quả của hai hệ thống hoán đổi được cho nhau: mỗi phép hoán vị đổi nhãn hai hệ thống cho toàn bộ truy vấn của một theorem rồi tính lại đúng thống kê quan sát. p hai phía là tỉ lệ phép hoán vị có trị tuyệt đối của chênh lệch không nhỏ hơn giá trị quan sát, tính cả trường hợp bằng nhau; với 10.000 phép hoán vị ngẫu nhiên, $p = (1 + \text{số lần cực trị})/10.001$ [21]. Unit test kiểm tra rằng hai hệ thống giống nhau cho $p = 1$, đảo vai trò hai hệ thống không đổi p, và kết quả khớp phép duyệt hết hoán vị trên ví dụ nhỏ. Nhóm phép so sánh chính (family of hypotheses) là 7 bước của chuỗi cải tiến trên Recall@10, hiệu chỉnh Holm [22] trong từng split; ablation hiệu chỉnh Holm trên 9 phép so sánh của từng chỉ số; các phân tích khác báo p chưa hiệu chỉnh. Bản trước của báo cáo dùng tỉ lệ đuôi của phân phối bootstrap làm p; cách tính đó sai khi chênh lệch bằng 0 và không ứng với một giả thuyết không rõ ràng nên đã được thay. Kết quả chất lượng là tất định: hai lần chạy B0 cho tệp kết quả có cùng mã md5.

CHƯƠNG 6

Kết quả

6.1 Chất lượng truy hồi

Bảng 6.1 trình bày chất lượng trên toàn bộ tập test, tìm trong toàn bộ 180.957 premise của corpus. V4 đạt Recall@10 bằng 0,243 trên `random`, gấp 2,4 lần B0, và MRR bằng 0,196, gấp 3 lần B0. Trên `novel_premises`, V4 đạt Recall@10 bằng 0,269 so với 0,127 của B0. V5 đạt 0,281 trên `random` và 0,280 trên `novel_premises`; Mục 6.2 phân tích mức tăng này kỹ hơn vì V5 được thiết kế sau khi xem tập test.

Bảng 6.1. Chất lượng truy hồi trên toàn bộ tập test, query-macro

	`random`						`novel_premises`		
Biến thể	R@1	R@5	R@10	R@50	MRR	nDCG@10	R@10	R@50	MRR
B0	0,018	0,070	0,100	0,176	0,065	0,061	0,127	0,211	0,090
B0t	0,021	0,083	0,124	0,224	0,078	0,073	0,155	0,272	0,106
V1	0,023	0,095	0,134	0,245	0,087	0,081	0,163	0,302	0,110
V2	0,023	0,101	0,147	0,270	0,093	0,087	0,183	0,335	0,124
V3	0,029	0,116	0,168	0,296	0,112	0,103	0,205	0,352	0,140
V2a	0,053	0,135	0,183	0,328	0,148	0,125	0,221	0,381	0,176
V4	0,072	0,184	0,243	0,373	0,196	0,169	0,269	0,417	0,211
V5	**0,073**	**0,198**	**0,281**	**0,508**	**0,217**	**0,188**	**0,280**	**0,509**	**0,215**

Ghi chú: 2.641 truy vấn `random`, 4.141 truy vấn `novel_premises`. Chữ đậm là giá trị cao nhất của từng cột; cấu hình đề xuất dùng chung là V4 (Mục 6.2).

Bảng 6.2 cho chênh lệch Recall@10 giữa các bước của chuỗi cải tiến. Trên `random`, 6 trên 7 bước có ý nghĩa ở mức 0,05 sau hiệu chỉnh Holm (p Holm 0,0007). Bước còn lại là tokenizer Lean sau khi đã tinh BM25: Recall@10 tăng 0,010 với p bằng 0,053, dù MRR của bước này tăng có ý nghĩa (p Holm 0,0095). Trên `novel_premises`, tokenizer Lean và bước từ V4 sang V5 đều có p Holm bằng 0,094, nên không có ý nghĩa ở mức 0,05.

Bảng 6.2. Chênh lệch Recall@10 giữa các bước cải tiến trên tập test, query-macro

	`random`		`novel_premises`	
So sánh	ΔRecall@10 [KTC 95%]	p Holm	ΔRecall@10 [KTC 95%]	p Holm
B0t so với B0	+0,023 [0,013; 0,033]	0,0007	+0,027 [0,019; 0,035]	0,0007
V1 so với B0t	+0,010 [0,0001; 0,022]	0,0527	+0,008 [0,0003; 0,016]	0,0940
V2 so với V1	+0,013 [0,006; 0,020]	0,0007	+0,020 [0,013; 0,027]	0,0007
V3 so với V2	+0,021 [0,010; 0,033]	0,0007	+0,022 [0,012; 0,032]	0,0007
V2a so với V2	+0,036 [0,029; 0,044]	0,0007	+0,039 [0,032; 0,045]	0,0007
V4 so với V2a	+0,060 [0,046; 0,074]	0,0007	+0,048 [0,038; 0,059]	0,0007
V5 so với V4	+0,038 [0,024; 0,053]	0,0007	+0,011 [−0,0005; 0,022]	0,0940
V5 so với B0	+0,180 [0,162; 0,200]	0,0001*	+0,153 [0,138; 0,169]	0,0001*

Ghi chú: KTC 95% và p hai phía theo Mục 5.5; p hiệu chỉnh Holm trên 7 bước của chuỗi trong từng split, giá trị nhỏ nhất có thể là $1/10.001$. Dấu * là p chưa hiệu chỉnh.

Trả lời RQ1. BM25 mặc định chưa đủ cho dữ liệu Lean: Recall@10 chỉ đạt 0,100. Tinh k_1, b và dùng qtf nhị phân thêm 0,023. Tokenizer Lean thêm 0,010 nhưng không có ý nghĩa sau hiệu chỉnh trên cả hai split. BM25F ba trường thêm 0,013 và có ý nghĩa, chủ yếu nhờ trường name: bỏ riêng trường namespace (V2-nons) chỉ làm Recall@10 giảm 0,001 trên `random`, khoảng tin cậy chứa 0 (Bảng 7.2), và giảm 0,002 trên `novel_premises` (p chưa hiệu chỉnh 0,030). Thông tin phụ thuộc có tác dụng lớn nhất. Bộ lọc accessible premise thêm 0,036 khi áp dụng cho V2. Bỏ cả popularity và PageRank làm Recall@10 giảm 0,018 (p Holm 0,008), còn giữ riêng một trong hai cho kết quả không phân biệt được với V4, nên chỉ kết luận được tác dụng của cả cặp. Bước từ V2a sang V4 (tăng 0,060) thêm cùng lúc reranker và hai đặc trưng, nên đó không phải tác dụng riêng của PageRank. Câu trả lời cho RQ1: BM25 mặc định chưa đủ; trường name và đặc biệt là thông tin phụ thuộc có tác dụng đo được, còn với tokenizer Lean (trên Recall@10) và trường namespace thì chưa có đủ bằng chứng.

Bối cảnh và so sánh với nhóm khác. Bảng C.4 ở Phụ lục C đặt số liệu của bài báo LeanDojo [3] cạnh kết quả của nhóm: hai dòng BM25 lấy từ Bảng 1 (benchmark cho Lean 3), dòng ReProver từ Bảng B ở Phụ lục D (một snapshot LeanDojo Benchmark 4 cũ hơn v10). Vì khác benchmark và snapshot, các con số chỉ cho thấy kết quả của nhóm nằm trong cùng khoảng giá trị với các hệ thống sparse retrieval đã công bố; chúng không dùng để xếp hạng hệ thống hay xác nhận tính đúng của pipeline. Tính đúng được kiểm tra bằng unit test, đối chiếu với cài đặt tham chiếu và với Lucene (Mục 6.5). So sánh với Nhóm 4 trên benchmark chung (`docs/benchmark_spec.md`) sẽ dùng cùng kiểm định và trộn kết quả bằng Reciprocal Rank Fusion với hằng số 60 [16]. Script trộn đã sẵn sàng nhưng chưa chạy trên kết quả thật, nên phần này chưa hoàn tất. [Bảng so sánh với Nhóm 4: chờ tệp kết quả và xác nhận đặc tả.]

6.2 Độ bền của kết quả V5

Vì V5 được thiết kế từ phân tích lỗi trên tập test `random`, nhóm kiểm tra mức tăng của nó theo ba cách: so với đối chứng V4-K240, so query-macro với theorem-macro, và đánh giá trên holdout (Bảng 6.3). V4-K240 lấy top 240 của BM25F, xấp xỉ số candidate của V5 trên `random` (239) và holdout (236) nhưng nhiều hơn V5 trên `novel_premises` (208), và vẫn dùng 13 đặc trưng. Chênh lệch giữa V5 và V4-K240 vì vậy là tác dụng gộp của các nguồn candidate theo lịch sử sử dụng và hai đặc trưng `file_usage`, `dir_usage`; không tách riêng được từng phần.

Bảng 6.3. Recall@10 của V4, V4-K240 và V5 trên hai tập test và holdout, theo query-macro và theorem-macro

Tập	Macro	V4	V4-K240	V5	V5 − V4	V5 − V4-K240
`random`	query	0,243	0,254	0,281	+0,038 [0,024; 0,053]	+0,027 [0,014; 0,041]
	theorem	0,359	0,372	0,393	+0,034 [0,017; 0,052]	+0,021 [0,004; 0,038]
`novel_premises`	query	0,269	0,276	0,280	+0,011 [−0,0005; 0,022]	+0,004 [−0,008; 0,016]
	theorem	0,372	0,377	0,359	−0,014 [−0,026; −0,002]	−0,019 [−0,031; −0,007]
Holdout	query	0,310	0,321	0,332	+0,022 [0,009; 0,035]	+0,011 [−0,002; 0,024]
	theorem	0,371	0,381	0,388	+0,017 [0,002; 0,032]	+0,007 [−0,007; 0,022]

Ghi chú: KTC 95% từ bootstrap theo cụm theorem. Holdout gồm 2.500 truy vấn của 1.138 theorem; số candidate của V4-K240 so với V5 xem Mục 5.

Trên tập test `random`, V5 hơn V4 0,038 và hơn V4-K240 0,027 ($p = 0{,}0002$) ở cả query-macro lẫn theorem-macro. Trên holdout, V5 hơn V4 0,022 (khoảng tin cậy [0,009; 0,035], $p = 0{,}001$), nên lợi ích so với V4 lặp lại trên dữ liệu chưa dùng để huấn luyện hay chọn cấu hình. Theo một phân rã mô tả, V4-K240 hơn V4 0,011 và V5 hơn V4-K240 thêm 0,011 (khoảng tin cậy [−0,002; 0,024], $p = 0{,}098$). Đây không phải phép tách tác động nhân quả độc lập, vì candidate pool, đặc trưng và quá trình huấn luyện tương tác với nhau. Như vậy V5 tốt hơn V4-K240 trên tập test `random`, tập đã dùng để thiết kế V5, nhưng trên holdout chưa có đủ bằng chứng cho thấy V5 tốt hơn V4-K240.

Trên `novel_premises`, kết luận phụ thuộc cách lấy trung bình. Theo query-macro, V5 hơn V4 0,011 (p Holm 0,094); theo theorem-macro, V5 kém V4 0,014 (khoảng tin cậy $[-0{,}026;\ -0{,}002]$, $p = 0{,}027$, p Holm 0,053). Với 536 theorem chỉ có một truy vấn, Recall@10 giảm từ 0,447 xuống 0,407, còn với 240 theorem có từ 5 truy vấn trở lên (2.525 trên 4.141 truy vấn) Recall@10 tăng từ 0,198 lên 0,225. Mức tăng của V5 tập trung ở các chứng minh dài.

Bảng 6.4. Recall trên unseen premise và seen premise, tập test `novel_premises`

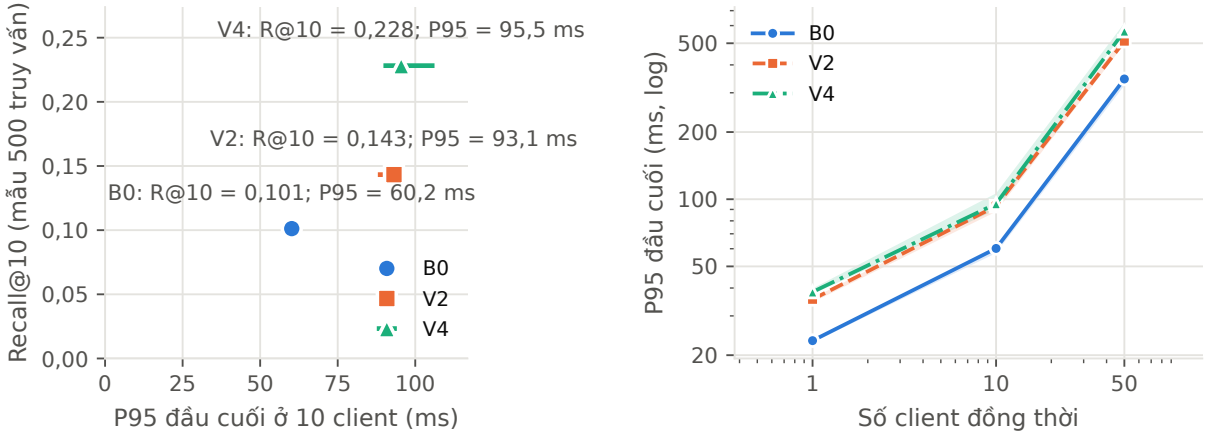
Hệ thống	R@10, unseen	R@50, unseen	R@10, seen	R@50, seen
B0	0,293	0,443	0,044	0,101
V2	0,356	0,603	0,092	0,205
V4	0,444	0,690	0,178	0,286
V4-K240	0,428	0,682	0,193	0,335
V5	0,359	0,603	0,231	0,451

Ghi chú: unseen premise chiếm 2.506 trên 9.735 cặp nhãn, thuộc 1.887 truy vấn và 1.093 trên 1.196 theorem; mỗi cột chỉ lấy trung bình trên các truy vấn có loại nhãn tương ứng.

Bảng 6.4 tách gold premise của `novel_premises` thành unseen premise và seen premise. V5 tăng Recall@10 trên seen premise từ 0,178 lên 0,231 nhưng giảm trên unseen premise từ 0,444 xuống 0,359 (khoảng tin cậy $[-0{,}100;\ -0{,}071]$). V4-K240 chỉ giảm 0,017, nên phần lớn mức giảm đến từ các nguồn candidate và đặc trưng dựa trên lịch sử sử dụng: chúng đẩy premise đã được dùng nhiều lên trên và chiếm chỗ của premise mới. Trên tập test `random`, với 517 truy vấn có unseen premise, Recall@10 trên unseen premise cũng giảm từ 0,410 xuống 0,340. V5 vì vậy đổi recall trên unseen premise lấy recall trên seen premise; với mục tiêu tìm premise mới, V4 phù hợp hơn.

6.3 Đánh đổi giữa chất lượng và độ trễ

Hình 6.1a đặt chất lượng và độ trễ của B0, V2 và V4 trên cùng một biểu đồ, đo trong đợt kiểm tra tải chính (chỉ mục đầy đủ, 10 client, 4 worker). P95 latency đầu cuối lần lượt là 60,2; 93,1 và 95,5 ms, QPS là 259, 166 và 130. Bước từ B0 sang V2 (BM25F ba trường với tokenizer Lean) làm P95 tăng 32,9 ms; bước từ V2 sang V4 chỉ thêm 2,4 ms nhưng nâng Recall@10 trên mẫu 500 truy vấn từ 0,143 lên 0,228. Phân rã thời gian trong một tiến trình (Bảng C.7, Phụ lục C) cũng cho thấy điều này: với V4, bước cộng điểm BM25F có trung vị 18,26 ms và chiếm 88% thời gian, còn bộ lọc, tính 13 đặc trưng và rerank lần lượt mất 0,85; 0,54 và 0,06 ms.



(a) Recall@10 (mẫu 500 truy vấn) và P95 ở 10 client

(b) P95 theo số client đồng thời (trục logarit)

Hình 6.1. Đợt kiểm tra tải chính, chỉ mục đầy đủ, dịch vụ 4 worker. Thanh ngang và vùng tô là khoảng giữa lần chạy nhanh nhất và chậm nhất trong 3 lần.

Bảng 6.5. V4, V4-K240 và V5 dưới tải trong cùng một phiên đo

Biến thể	Client	P95 (ms) [nhỏ nhất; lớn nhất]	QPS [nhỏ nhất; lớn nhất]	Yêu cầu lỗi
V4	1	40,5 [40,4; 43,4]	40,2 [37,5; 40,4]	0 / 1.200
	10	103,2 [99,1; 118,8]	150,1 [116,6; 157,8]	0 / 1.200
	50	622,0 [611,5; 654,0]	132,4 [126,2; 137,5]	0 / 1.200
V4-K240	1	44,1 [43,7; 45,3]	35,4 [34,9; 36,5]	0 / 1.200
	10	124,6 [98,5; 147,5]	121,8 [103,0; 142,8]	0 / 1.200
	50	650,6 [607,6; 654,3]	129,5 [123,7; 131,6]	0 / 1.200
V5	1	44,7 [40,5; 47,1]	35,4 [34,8; 39,4]	0 / 1.200
	10	116,7 [113,1; 138,7]	127,6 [3,3; 134,4]	1 / 1.200
	50	726,7 [685,2; 727,4]	116,0 [107,6; 118,3]	0 / 1.200

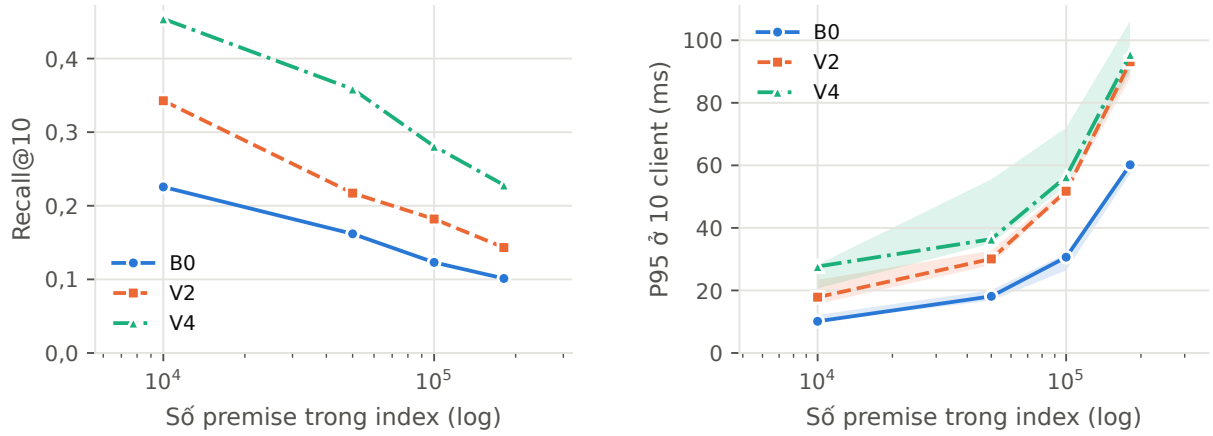
Ghi chú: một server 4 worker nạp cả ba cấu hình, thứ tự đo xoay vòng giữa 3 lần lặp, mỗi lần 400 yêu cầu ở mỗi mức client. P95 tính trên các yêu cầu thành công; QPS tính trên toàn bộ thời gian chạy, kể cả thời gian chờ yêu cầu bị timeout. Mỗi ô là trung vị [nhỏ nhất; lớn nhất] của 3 lần.

V5 không có mặt trong đợt đo tải chính, nên nhóm đo riêng V4, V4-K240 và V5 trong cùng một phiên, thứ tự đo xoay vòng giữa 3 lần lặp (Bảng 6.5). Ở 10 client, P95 của V5 là 116,7 ms so với 103,2 ms của V4; khoảng biến thiên của hai hệ thống chồng nhau và ba lần lặp chưa đủ để ước lượng khoảng tin cậy, nên nhóm không kết luận V5 tương đương V4 ở mức tải này. Ở 50 client, khi máy đã bão hòa CPU, P95 tăng từ 622,0 lên 726,7 ms và QPS giảm từ 132 xuống 116, phù hợp với việc phải tính đặc trưng cho khoảng 240 candidate; một phiên đo trước đó cho kết quả cùng chiều (QPS 131 so với 108). Một yêu cầu V5 ở 10 client bị timeout sau 120 giây, chưa rõ nguyên nhân: P95 của lần đo đó chỉ tính các yêu cầu thành công (138,7 ms), còn QPS tính cả thời gian chờ nên chỉ đạt 3,3. Độ tin cậy của V5 dưới tải vì vậy chưa được kiểm chứng đủ.

Trả lời RQ2 và câu hỏi trung tâm. Kết hợp BM25F với các đặc trưng cấu trúc làm tăng Recall@10 thêm 0,096 trên toàn bộ tập test `random` (từ V2 lên V4), với chi phí 2,4 ms P95 ở 10 client trong đợt đo chính. Các đặc trưng cấu trúc cải thiện recall thật sự, và chi phí của chúng nhỏ so với chi phí của chính giai đoạn BM25F. Mở rộng candidate pool như V5 có chi phí rõ hơn khi tải cao và, như Mục 6.2 cho thấy, không cải thiện đồng đều. Nếu cần giảm độ trễ, điểm cần tối ưu là giai đoạn 1 chứ không phải reranker.

6.4 Ảnh hưởng của kích thước kho

Hình 6.2 cho thấy chất lượng và độ trễ khi chỉ mục tăng từ 10K lên 181K premise. Mọi mức đều chứa đủ 870 gold premise của mẫu, nên thiết kế này đo tác động của premise gây nhiễu, không mô phỏng thư viện lớn dần theo thời gian; việc đổi corpus cũng đổi idf và độ dài trung bình của trường. Recall@10 giảm đều: B0 từ 0,226 xuống 0,101, V2 từ 0,343 xuống 0,143 và V4 từ 0,454 xuống 0,228. P95 ở 10 client tăng từ 10,1 lên 60,2 ms với B0 và từ 27,6 lên 95,5 ms với V4, vì posting list của các token phổ biến dài ra; chỉ mục ba trường tăng gần tuyến tính từ 2,1 MB lên 26,3 MB.



(a) Recall@10 theo số premise trong chỉ mục

(b) P95 ở 10 client theo số premise

Hình 6.2. Chất lượng và độ trễ theo kích thước chỉ mục (mẫu 500 truy vấn; vùng tô là khoảng giữa lần nhanh nhất và chậm nhất trong 3 lần).

6.5 Độ trễ dưới tải đồng thời

Hình 6.1b cho kết quả kiểm tra tải trên chỉ mục đầy đủ với 4 worker. Từ 1 lên 10 client, QPS tăng từ 3 đến 3,5 lần và P95 tăng khoảng 2,5 lần (V4 từ 38,4 lên 95,5 ms). Ở 50 client cả ba hệ thống đều bão hòa: QPS gần như không đổi và P95 lên 345,9 đến 566,8 ms vì các yêu cầu phải xếp hàng. Trong 43.200 yêu cầu có 4 yêu cầu lỗi, đều phía client; khoảng biến thiên tương đối của P95 giữa các lần lặp có trung vị 15% và lớn nhất 56%.

Thêm worker có giúp không? Với 4, 8 và 12 worker và thêm mức 100 client (Bảng C.8, Phụ lục C), thông lượng gần như không đổi (B0 khoảng 230 đến 260 QPS, V4 khoảng 140 đến 160 QPS) trong khi RAM của server tăng từ 3,4 lên 9,8 GB. Ngay với 4 worker, các worker đã dùng 8 đến 9,5 trên 12 luồng logic, và tạo tải bằng bốn tiến trình client cũng không tăng QPS. Trên máy này dịch vụ bị giới hạn bởi CPU: muốn tăng thông lượng phải giảm chi phí CPU mỗi truy vấn hoặc thêm máy (chưa đo), còn thêm worker chỉ tốn bộ nhớ.

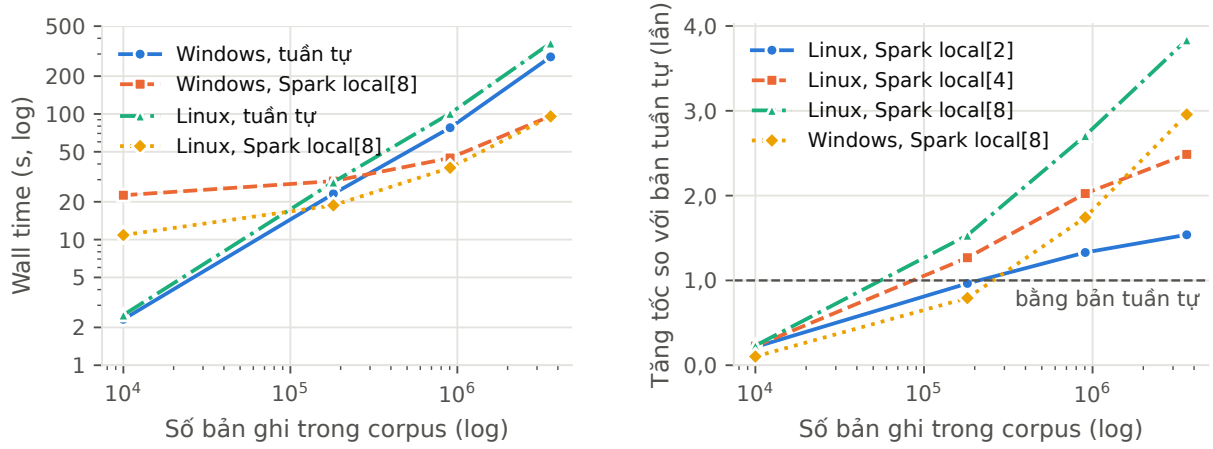
Đối chiếu với Lucene. Với BM25 của Lucene 10.5 qua Anserini 2.3.0 trên cùng trường và cùng tập test (Bảng C.6, Phụ lục C), khi nhận đúng các token của B0 cùng k_1 và b, Lucene cho các chỉ số rất gần B0 (MRR 0,0657 so với 0,0654); top 10 trùng trung bình 98,9% phần tử và trùng nguyên tập ở 89,6% truy vấn, phần lệch còn lại phù hợp với việc Lucene nén độ dài tài liệu. Giữ nguyên k_1 và b, bộ phân tích tiếng Anh làm Recall@10 giảm từ 0,100 xuống 0,089 vì bỏ stopword và rút gọn từ làm hỏng tên Lean; tham số mặc định của Anserini làm giảm thêm xuống 0,076. Lucene mất 2,4 ms mỗi truy vấn so với 13,2 ms của B0 đo cùng quy tắc, tức nhanh hơn khoảng 5,5 lần; đây là trung bình khi chạy một thread với cấu hình B0, chưa phải bằng chứng về BM25F ba trường hay P95 của API sau khi tích hợp Lucene.

6.6 Xây chỉ mục theo lô

Đường xây chỉ mục được đo qua hai đợt. Đợt đầu dùng engine RDD trên chỉ mục B0 với corpus tới 904.785 bản ghi trên Windows; bản tuần tự nhanh nhất ở mọi kích thước (ở 905K bản ghi, 17,8 giây so với 41,0 giây của local[4]), vì driver đọc toàn bộ dữ liệu rồi mới phân phối, mỗi posting là một cặp khóa và giá trị Python phải tuần tự hóa, và job có hai lượt shuffle lớn. Đợt thứ hai dùng engine Arrow cho chỉ mục B0 và chỉ mục ba trường, với corpus tới 3,6 triệu bản ghi, trên Windows và trong container Linux trên cùng phần cứng; cả 194 lần chạy đều thành công và mọi cách xây cho cùng mã SHA-1 của từng tệp chỉ mục.

Bảng 6.6. Thời gian xây chỉ mục (giây, trung vị của 3 lần, gồm cả thời gian khởi động Spark). Tăng tốc là thời gian tuần tự chia cho thời gian Spark local[8] trên cùng nền tảng; hai mức cuối là corpus nhân bản 5 và 20 lần

Nền tảng và chỉ mục	Cách xây	10K	181K	905K	3,6 triệu
Windows, ba trường	Tuần tự	2,3	23,1	77,7	284,7
	Spark local[8]	22,6	29,2	44,6	96,3
	Tăng tốc	0,10	0,79	**1,74**	**2,96**
Linux, ba trường	Tuần tự	2,5	28,7	101,1	366,8
	Spark local[8]	10,9	18,7	37,3	95,6
	Tăng tốc	0,23	**1,53**	**2,71**	**3,84**
Windows, B0	Tuần tự	1,2	5,9	17,0	58,9
	Spark local[8]	22,4	23,3	27,2	37,0
	Tăng tốc	0,05	0,25	0,63	**1,59**
Linux, B0	Tuần tự	1,0	6,7	20,4	71,5
	Spark local[8]	10,4	12,3	17,4	30,9
	Tăng tốc	0,10	0,55	**1,17**	**2,31**



(a) Thời gian xây chỉ mục ba trường

(b) Tăng tốc so với bản tuần tự, chỉ mục ba trường

Hình 6.3. Xây chỉ mục bằng engine Arrow theo kích thước corpus (trục hoành logarit; hai mức cuối là corpus nhân bản 5 và 20 lần, chỉ dùng để đo thời gian, bộ nhớ và độ lệch).

Điểm giao nhau. Ở kích thước nhỏ, chi phí cố định của Spark (khởi động phiên khoảng 4 giây, tạo tiến trình Python cho mỗi task) chiếm phần lớn thời gian, nhất là trên Windows, nơi PySpark không fork được worker. Khi corpus lớn lên, phần tính toán lấn át chi phí cố định (Bảng 6.6, Hình 6.3). Trên corpus mathlib4 thật (181K premise), Spark local[8] chỉ nhanh hơn bản tuần tự với chỉ mục ba trường trên Linux (1,53 lần); trên corpus nhân bản 3,6 triệu bản ghi, nhanh hơn 3,84 lần trên Linux và 2,96 lần trên Windows. Các mức này đo bằng Spark chế độ local trên một máy 6 nhân, nên chứng minh lợi ích của song song hóa cục bộ chứ chưa phải khả năng mở rộng trên cụm, và corpus nhân bản giữ nguyên số khóa nên thuận lợi cho engine Arrow.

Số thread, bộ nhớ và độ ổn định. Ở 3,6 triệu bản ghi trên Linux, thời gian giảm từ 238,5 giây (local[2]) xuống 95,6 giây (local[8]) còn RAM đỉnh tăng từ mức của bản tuần tự (5,0 GB) lên 8,2 GB. Ở 58 trên 64 cấu hình, các lần lặp chênh nhau không quá 5% trung vị. Hai nền tảng chạy Python khác phiên bản nên chỉ so sánh trong cùng một nền tảng.

CHƯƠNG 7

Ablation, phân tích lỗi và hạn chế

7.1 Ablation của reranker

Mỗi biến thể trong Bảng 7.1 bỏ đúng một nhóm đặc trưng hoặc đổi đúng một tham số của V4, rồi huấn luyện lại reranker trên cùng dữ liệu. Giả thuyết và quy tắc kết luận được ghi trong `docs/ablation_plan.md`: thành phần có tác dụng nếu p sau hiệu chỉnh Holm nhỏ hơn 0,05; ngược lại kết luận là không phân biệt được, không phải là vô dụng.

Bảng 7.1. Ablation của reranker V4 trên tập test `random`

Thay đổi so với V4	ΔRecall@10 [KTC 95%]	p Holm	ΔMRR [KTC 95%]	p Holm
Tắt bộ lọc accessible premise	−0,051 [−0,061; −0,042]	0,0009	−0,066 [−0,076; −0,057]	0,0009
Bỏ điểm và hạng BM25	−0,036 [−0,047; −0,026]	0,0009	−0,046 [−0,059; −0,035]	0,0009
Bỏ độ phổ biến và PageRank	−0,018 [−0,029; −0,007]	0,0077	−0,015 [−0,023; −0,008]	0,0009
Bỏ độ trùng ký hiệu	−0,009 [−0,017; −0,001]	0,1320	−0,008 [−0,013; −0,004]	0,0009
Bỏ loại khai báo	−0,001 [−0,007; 0,005]	1,0000	−0,008 [−0,014; −0,004]	0,0010
Bỏ cùng tệp, cùng thư mục	+0,000 [−0,007; 0,008]	1,0000	−0,010 [−0,016; −0,004]	0,0010
Bỏ namespace	−0,000 [−0,004; 0,003]	1,0000	−0,000 [−0,002; 0,002]	0,8770
50 candidate thay vì 100	−0,009 [−0,016; −0,001]	0,1320	−0,007 [−0,010; −0,003]	0,0033
200 candidate thay vì 100	+0,010 [0,003; 0,016]	0,0192	+0,006 [0,002; 0,010]	0,0060

Ghi chú: 2.641 truy vấn; V4 đầy đủ có Recall@10 = 0,243, MRR = 0,196. Δ là biến thể trừ V4. Khoảng tin cậy từ bootstrap theo cụm theorem; p hai phía từ kiểm định hoán vị ghép cặp theo cụm theorem, hiệu chỉnh Holm trên 9 phép so sánh của từng metric.

Bảng 7.2. Đối chứng cho trường namespace và hai đặc trưng phụ thuộc

	Validation		Test `random`			
Cấu hình	R@10	MRR	R@10	MRR	ΔR@10	ΔMRR
V2	0,172	0,115	0,147	0,093		
V2-nons	0,172	0,114	0,146	0,093	−0,001 [−0,005; 0,002]	−0,000 [−0,001; 0,001]
V4	0,273	0,220	0,243	0,196		
V4-poponly	0,273	0,220	0,244	0,196	+0,002 [−0,0001; 0,004]	+0,000 [−0,001; 0,001]
V4-pronly	0,276	0,217	0,244	0,191	+0,001 [−0,006; 0,008]	−0,004 [−0,009; 0,0001]

Ghi chú: V2-nons bỏ trường namespace khỏi BM25F; V4-poponly và V4-pronly chỉ giữ popularity hoặc PageRank. Δ là cấu hình rút gọn trừ cấu hình đầy đủ trên tập test, kèm KTC 95%.

Bộ lọc accessible premise là thành phần có giá trị nhất; tiếp theo là điểm của giai đoạn 1 và cặp đặc trưng phụ thuộc. Sau hiệu chỉnh Holm, bỏ độ trùng ký hiệu (−0,009) và giảm xuống 50 candidate (−0,009) không còn có ý nghĩa với Recall@10 (p Holm bằng 0,132), dù vẫn có ý nghĩa với MRR. Đặc trưng vị trí và loại khai báo không làm thay đổi Recall@10 nhưng làm giảm MRR có ý nghĩa khi bị bỏ, nghĩa là chúng giúp đưa gold premise lên đầu danh sách mà không đưa thêm gold premise vào top 10. Đặc trưng namespace của reranker không có tác dụng đo được ở cả hai chỉ số. Tăng số candidate lên 200 làm Recall@10 tăng 0,010 (p Holm bằng 0,019); V4 giữ 100 candidate vì giá trị này đã được chốt trước khi chạy tập test.

Bảng 7.2 tách thêm hai thành phần mà ablation trên chưa tách. Bỏ trường namespace khỏi BM25F, một can thiệp khác với bỏ đặc trưng namespace của reranker, không tạo khác biệt có ý nghĩa trên `random`. Giữ

riêng popularity hoặc riêng PageRank cho Recall@10 trong khoảng 0,002 quanh V4 với khoảng tin cậy chứa 0.

7.2 Ablation của đường xây chỉ mục

Hai engine Spark. Trên cùng chỉ mục B0 với Spark local[4] (Bảng C.9, Phụ lục C), engine Arrow nhanh hơn engine RDD 1,7 lần ở 905K và 3,0 lần ở 3,6 triệu bản ghi, và cần ít RAM hơn 3 đến 6 lần vì driver của engine RDD phải giữ toàn bộ dữ liệu đầu vào. Dung lượng shuffle sau nén của hai engine tương đương (183 MB và 204 MB ở 3,6 triệu bản ghi); khác biệt nằm ở số bản ghi Python phải tuần tự hóa: khoảng 67 triệu cặp khóa và giá trị ở engine RDD so với khoảng 0,5 triệu bản ghi ở engine Arrow.

Kết hợp cục bộ và tách khóa nóng. Với engine RDD ở 905K bản ghi, bỏ kết hợp cục bộ làm dữ liệu shuffle tăng 2,5 lần (từ 52,1 lên 130,7 MB) và thời gian tăng từ 41,0 lên 58,3 giây. Tách khóa nóng thành 4 phần giảm thời gian của task reduce chậm nhất từ 6,4 xuống 4,2 giây nhưng cần thêm hai pha, nên tổng thời gian tăng lên 53,5 giây. Ở quy mô này độ lệch chưa đủ lớn để việc tách khóa có lợi.

Cách mã posting list. Với chỉ mục B0 khi tắt bộ đệm, dung lượng posting là 12,8 MB với số nguyên 32 bit, 4,1 MB với VB và 2,6 MB với gamma; P50 truy vấn tương ứng là 12,7; 16,9 và 137,7 ms. VB nhỏ hơn 3,1 lần so với số nguyên 32 bit và chỉ chậm hơn 4 ms, còn gamma chậm hơn VB khoảng 8 lần vì giải mã từng bit bằng Python. Khi bật bộ đệm như cấu hình mặc định, cả ba có trung vị độ trễ từ 12,9 đến 13,4 ms và cho cùng kết quả truy hồi.

7.3 Phân tích lỗi

Trên tập test `random`, V4 không đưa được 4.369 trong 5.483 cặp (truy vấn, gold premise) vào top 10. Bảng 7.3 phân loại tự động các cặp này; một cặp có thể thuộc nhiều nhóm. Phân tích này được làm trên tập test và đã dẫn tới thiết kế của V5 (Mục 5.3).

Bảng 7.3. Phân loại tự động 4.369 cặp (truy vấn, gold premise) mà V4 bỏ lỡ ở top 10 trên tập test `random`; một cặp có thể thuộc nhiều nhóm

Nhóm lỗi	Số cặp	Tỉ lệ
Gold premise nằm ngoài 100 candidate của giai đoạn 1	3.613	82,7%
Truy vấn có nhiều gold premise	2.430	55,6%
Premise tổng quát, thuộc 1% premise được dùng nhiều nhất	1.352	30,9%
Premise không có từ nào chung với truy vấn	1.080	24,7%
Premise là định nghĩa, instance hoặc structure	706	16,2%
Truy vấn dài hơn 160 token	442	10,1%
Không thuộc nhóm nào ở trên	336	7,7%

Phát hiện quan trọng nhất là 82,7% cặp bị bỏ lỡ nằm ngoài 100 candidate của giai đoạn 1, nên reranker không thể cứu được. Các premise tổng quát như bổ đề về $\leq$, $+$ hay $\in$ được dùng ở rất nhiều nơi nhưng ít trùng từ với một proof state cụ thể, và gần một phần tư premise bị bỏ lỡ không có từ nào chung với truy vấn. Đây là những trường hợp dense retrieval có khả năng bổ sung tốt. Theo lát cắt, V4 tốt hơn B0 ở mọi lĩnh vực có ít nhất 30 truy vấn; mức tăng Recall@10 lớn nhất ở LinearAlgebra (0,201) và FieldTheory (0,163), nhỏ nhất ở Combinatorics (0,049) và NumberTheory (0,059). Truy vấn ngắn từ 20 token trở xuống dễ nhất với Recall@10 bằng 0,378, truy vấn dài hơn 160 token khó nhất với 0,169, và truy vấn cần từ 5 premise trở lên chỉ đạt 0,120.

Bảng 7.4 theo dõi ví dụ xuyên suốt qua từng biến thể. Gold premise `mul_assoc` không có từ nào chung với proof state ngoài các ký hiệu * và =, nên B0 đến V4 đều không đưa nó vào 100 candidate; các biến thể này trả về những bổ đề về `invOf` có tên gần với proof state. V5 đưa nó vào candidate pool qua nguồn theo tệp, vì một theorem train A khai báo trước trong cùng tệp đã dùng nó, và reranker xếp nó ở hạng 2; `mul_assoc` cũng là premise được dùng nhiều thứ tư trong train A. Ví dụ này minh họa cơ chế, không phải bằng chứng định lượng; Mục 6.2 cho thấy cùng cơ chế đó làm giảm recall trên unseen premise.

Bảng 7.4. Ví dụ xuyên suốt: truy vấn validation `random:val:36:0`, gold premise `mul_assoc`. Hạng lấy từ kết quả top 100 của từng biến thể; ba kết quả đầu theo thứ tự hạng

Biến thể	Thay đổi	Hạng	Ba kết quả đầu
B0	BM25, tokenizer mặc định	ngoài top 100	`Invertible.congr`; `invOf_add_invOf`; `invOf_sub_invOf`
V2	BM25F ba trường, tokenizer Lean	ngoài top 100	`Invertible.congr`; `invOf_add_invOf`; `invOf_sub_invOf`
V2a	V2 và bộ lọc accessible premise	ngoài top 100	`Invertible`; `invOf_mul_self_assoc`; `invOf_mul_self_assoc'`
V4	V2a và reranker 13 đặc trưng	ngoài top 100	`invOf_mul_self_assoc`; `mul_invOf_self`; `invOf_mul_self`
V5	V4 và candidate từ lịch sử sử dụng	2	`invOf_mul_self`; `mul_assoc`; `one_mul`

Tạo candidate hay xếp hạng. Bảng 7.5 tách phần recall bị mất thành candidate loss và ranking gap. Với V4, candidate loss (0,603 trên `random`) lớn gấp khoảng bốn lần ranking gap (0,154), đúng như phân tích lỗi ở trên. Với V5, candidate recall tăng lên 0,650 nhưng ranking gap tăng lên 0,369, xấp xỉ candidate loss (0,350): phần lớn gold premise mới vào candidate pool vẫn bị reranker tuyến tính xếp ngoài top 10. Trong các gold premise nằm trong candidate pool của V5 trên `random`, 55,6% đến từ BM25F, 23,1% từ nguồn theo tệp, 10,9% từ nguồn theo thư mục và 10,3% từ nguồn phổ biến. Như vậy reranker trở thành một hướng cải thiện quan trọng ngang với bước tạo candidate, nhưng bước tạo candidate vẫn bỏ lỡ khoảng 35% gold premise.

Bảng 7.5. Candidate recall, Oracle@10 và ranking gap, query-macro

Tập	Hệ thống	Candidate	Candidate recall	Oracle@10	R@10	Ranking gap	R@50
`random`	V4	100	0,397	0,397	0,243	0,154	0,373
	V4-K240	240	0,485	0,485	0,254	0,232	0,419
	V5	239	0,650	0,650	0,281	0,369	0,508
`novel_premises`	V4	100	0,452	0,452	0,269	0,183	0,417
	V4-K240	240	0,536	0,536	0,276	0,260	0,449
	V5	208	0,644	0,644	0,280	0,364	0,509
Holdout	V4	100	0,452	0,452	0,310	0,142	0,432
	V4-K240	240	0,541	0,541	0,321	0,219	0,482
	V5	236	0,666	0,666	0,332	0,333	0,545

Ghi chú: định nghĩa ở Mục 5.5; Oracle@50 trùng candidate recall đến ba chữ số nên không ghi.

Gán nhãn thủ công. Ngoài phân loại tự động, nhóm đã lấy mẫu phân tầng 30 trường hợp thất bại kèm proof state, gold premise, hạng của nó và ba kết quả đầu của V4 (`results/figures/failure_cases.md`). Kế hoạch là hai thành viên gán nhãn độc lập theo cùng bảng nhóm lỗi, rồi đối chiếu và báo tỉ lệ đồng thuận

thô; việc gán nhãn này chưa được thực hiện. [Hai thành viên gán nhãn 30 trường hợp; điền tỉ lệ đồng thuận và các nhóm lỗi chính.]

7.4 Một lỗi được phát hiện nhờ phân tích lỗi

Phiên bản đầu của bộ lọc accessible premise so dòng *bắt đầu* của premise cùng tệp với dòng bắt đầu của theorem. Vì vị trí của một khai báo trong dữ liệu tính cả docstring và thuộc tính đứng trước nó, chính theorem đang chứng minh có thể có dòng bắt đầu nhỏ hơn dòng bắt đầu được ghi cho truy vấn và lọt qua bộ lọc. Lỗi này xảy ra ở 1.111 trên 2.641 truy vấn của tập test, tức 42%. Nhóm phát hiện lỗi khi thấy chính theorem đang chứng minh xuất hiện trong danh sách candidate. Bộ lọc được sửa để so dòng kết thúc, có thêm test hồi quy kiểm tra rằng theorem không bao giờ là accessible premise của chính nó và mọi gold premise của tập test đều là accessible premise. V2a và V4 đã được huấn luyện và đánh giá lại; mọi số liệu trong báo cáo là sau khi sửa.

7.5 Hạn chế và threats to validity

Phạm vi của benchmark. Benchmark chỉ tính premise có trong corpus, nên recall báo cáo cao hơn recall mà bộ chứng minh gặp thực tế; kết quả cũng chỉ đại diện cho 44,1% theorem trong tập test `random` còn truy vấn sau lọc. So sánh giữa các biến thể vẫn công bằng vì dùng cùng định nghĩa và cùng tập truy vấn.

Tập test tham gia thiết kế. V5 được thiết kế từ phân tích lỗi trên tập test `random`, nên kết quả của V5 trên tập test có được sau khi đã xem lỗi trên chính tập đó. Holdout lấy từ phần train B chưa dùng để huấn luyện reranker, cùng snapshot và có ít truy vấn mỗi theorem hơn tập test, nên không chứng minh khả năng tổng quát hóa sang phiên bản thư viện khác. Tham số của giai đoạn 1 được chọn trên một lưới giới hạn; với V2, cấu hình được chọn thấp hơn điểm tốt nhất trên validation 0,0005 theo một quy tắc không định trước.

Tương quan và nhiều phép so sánh. Các truy vấn của cùng một theorem không độc lập: bootstrap theo cụm cho khoảng tin cậy rộng hơn bootstrap theo truy vấn, và theorem-macro đổi dấu chênh lệch giữa V4 và V5 trên `novel_premises`. Kiểm định hoán vị giả định các theorem hoán đổi được độc lập. Nhóm phép so sánh chính và ablation đã hiệu chỉnh Holm; các so sánh phụ thì chưa, nên một kết quả phụ có p gần 0,05 chỉ là gợi ý. Ablation của V4, V4-poponly và V4-pronly chỉ chạy trên `random`; V2-nons có cả hai split, còn V4-K240 có cả hai split và holdout.

Prior tĩnh. Popularity và PageRank tính một lần trên train A, không theo vị trí của từng truy vấn. Điều này hợp lệ với benchmark tĩnh nhưng không mô phỏng việc viết một định lý mới tại một thời điểm.

Một máy, chế độ local, corpus nhân bản. Mọi phép đo thời gian thực hiện trên một máy 6 nhân, kể cả thí nghiệm Linux trong container, nên chưa phản ánh chi phí mạng và điều phối của một cụm; chương trình tạo tải cũng chạy cùng máy với server. Corpus gấp 5 và gấp 20 lần giữ nguyên phân phối term và số khóa, nên posting list dài và đều hơn dữ liệu thật; trên một corpus thật lớn cỡ đó, mức giảm bản ghi shuffle của engine Arrow sẽ nhỏ hơn.

Truy vết phiên bản mã. Các bản ghi lần chạy ghi phiên bản mã là `uncommitted` kèm mã băm của mã nguồn, vì kho mã chưa được commit tại thời điểm chạy. Bảng và hình của báo cáo đọc các lần chạy đã ghim trong `results/figures/report_manifest.json` và `report_records.json`; khi commit, các lần chạy cũ sẽ không được sửa hồi tố mà các lần chạy lại sẽ ghi mã commit đầy đủ.

CHƯƠNG 8

Kết luận và hướng phát triển

Với RQ1, BM25 mặc định chưa đủ cho dữ liệu Lean. Tinh tham số, trường name và đặc biệt là thông tin phụ thuộc có tác dụng đo được; với tokenizer Lean (trên Recall@10) và trường namespace thì chưa có đủ bằng chứng, còn popularity và PageRank chỉ kết luận được theo cặp. Với RQ2, kết hợp BM25F với các đặc trưng cấu trúc (V4) đưa Recall@10 từ 0,100 lên 0,243 trên tập test `random` và từ 0,127 lên 0,269 trên `novel_premises`; trong đợt đo tải chính, P95 latency ở 10 client tăng từ 60,2 lên 95,5 ms, phần lớn do giai đoạn BM25F, còn các đặc trưng cấu trúc chỉ tốn khoảng 2 ms. V4 là lựa chọn cân bằng giữa chất lượng và độ trễ. V5 bổ sung candidate từ lịch sử sử dụng, nâng Recall@10 trên `random` lên 0,281 và hơn V4 trên holdout (0,332 so với 0,310); tuy nhiên trên holdout chưa có đủ bằng chứng cho thấy V5 tốt hơn V4-K240, V5 làm giảm recall trên unseen premise của `novel_premises`, và có chi phí rõ hơn khi tải cao. V5 vì vậy là một đánh đổi có lợi cho premise quen thuộc. Về phục vụ, dịch vụ bị giới hạn bởi CPU của máy; BM25 của Lucene cho kết quả rất gần B0 và nhanh hơn khoảng 5,5 lần khi chạy một thread. Về khía cạnh dữ liệu lớn, chỉ mục xây bằng Spark trùng từng byte với bản tuần tự. Engine RDD ban đầu không nhanh hơn bản tuần tự ở kích thước nào; sau khi viết lại với lô Arrow và gộp posting theo partition, Spark local[8] nhanh hơn bản tuần tự 1,53 lần trên corpus thật 181K (Linux, chỉ mục ba trường) và 3,84 lần trên corpus nhân bản 3,6 triệu bản ghi, đều trên một máy. Cách tổ chức job, đặc biệt là gộp dữ liệu trước khi shuffle, quyết định lợi ích của xử lý song song.

Sản phẩm bàn giao gồm đặc tả benchmark, pipeline trích xuất có kiểm tra chất lượng, chỉ mục đảo tái tạo được kèm manifest, dịch vụ truy vấn qua API và CLI, baseline BM25, reranker cấu trúc, các script đánh giá, ablation và phân tích lỗi, README và kịch bản demo. Các bảng và hình của báo cáo được sinh bằng script từ kết quả thô và các lần chạy đã ghim; nguồn của từng bảng, từng hình và mã SHA-256 của đầu vào ghi trong `tables/SOURCES.md` và `figures/SOURCES.md`. Các việc còn phải do thành viên thực hiện gồm đối chiếu thủ công 20 truy vấn kiểm tra, gán nhãn 30 trường hợp thất bại, chốt phân công và so sánh với Nhóm 4.

Nhóm đề xuất năm hướng tiếp theo. Thứ nhất, trộn kết quả với hệ thống dense retrieval của Nhóm 4 bằng Reciprocal Rank Fusion thành hybrid retrieval, để tìm các premise không có từ chung với truy vấn và các unseen premise. Thứ hai, thay reranker tuyến tính bằng một mô hình xếp hạng phi tuyến, chọn trên validation và kiểm tra trên một holdout mới, vì Oracle@10 của V5 là 0,650 trong khi Recall@10 mới đạt 0,281; mô hình này cần giữ được recall trên unseen premise. Thứ ba, thay giai đoạn 1 bằng Lucene hoặc cài đặt các kỹ thuật dừng sớm như MaxScore và WAND [23], rồi đo lại P95 của API. Thứ tư, chạy thí nghiệm xây chỉ mục trên cụm nhiều máy với corpus thật lớn hơn. Thứ năm, tích hợp với một bộ chứng minh Lean để đo ảnh hưởng của chất lượng truy hồi tới tỉ lệ chứng minh thành công.

Tài liệu tham khảo

[1] L. de Moura, S. Ullrich. The Lean 4 Theorem Prover and Programming Language. *Automated Deduction, CADE 28*, Lecture Notes in Computer Science 12699, Springer, 2021.

[2] The mathlib Community. The Lean Mathematical Library. *Proceedings of the 9th ACM SIGPLAN International Conference on Certified Programs and Proofs (CPP)*, 2020.

[3] K. Yang, A. Swope, A. Gu, R. Chalamala, P. Song, S. Yu, S. Godil, R. Prenger, A. Anandkumar. LeanDojo: Theorem Proving with Retrieval-Augmented Language Models. *Advances in Neural Information Processing Systems 36 (NeurIPS), Datasets and Benchmarks Track*, 2023.

[4] LeanDojo documentation. `https://leandojo.readthedocs.io/`. Truy cập tháng 10/2026.

[5] S. Robertson, H. Zaragoza. The Probabilistic Relevance Framework: BM25 and Beyond. *Foundations and Trends in Information Retrieval* 3(4), 2009.

[6] C. D. Manning, P. Raghavan, H. Schütze. *Introduction to Information Retrieval*. Cambridge University Press, 2008.

[7] F. Scholer, H. E. Williams, J. Yiannis, J. Zobel. Compression of Inverted Indexes for Fast Query Evaluation. *Proceedings of the 25th ACM SIGIR Conference*, 2002.

[8] J. Meng, L. C. Paulson. Lightweight Relevance Filtering for Machine-Generated Resolution Problems. *Journal of Applied Logic* 7(1), 2009.

[9] G. Irving, C. Szegedy, A. A. Alemi, N. Eén, F. Chollet, J. Urban. DeepMath: Deep Sequence Models for Premise Selection. *Advances in Neural Information Processing Systems 29 (NeurIPS)*, 2016.

[10] M. Mikuła và cộng sự. Magnushammer: A Transformer-Based Approach to Premise Selection. *International Conference on Learning Representations (ICLR)*, 2024. arXiv:2303.04488.

[11] J. Dean, S. Ghemawat. MapReduce: Simplified Data Processing on Large Clusters. *6th Symposium on Operating Systems Design and Implementation (OSDI)*, 2004.

[12] M. Zaharia và cộng sự. Resilient Distributed Datasets: A Fault-Tolerant Abstraction for In-Memory Cluster Computing. *9th USENIX Symposium on Networked Systems Design and Implementation (NSDI)*, 2012.

[13] Apache Spark documentation. `https://spark.apache.org/docs/latest/`. Truy cập tháng 10/2026.

[14] J. Lin, C. Dyer. *Data-Intensive Text Processing with MapReduce*. Morgan & Claypool, 2010.

[15] L. Page, S. Brin, R. Motwani, T. Winograd. The PageRank Citation Ranking: Bringing Order to the Web. Technical Report, Stanford InfoLab, 1999.

[16] G. V. Cormack, C. L. A. Clarke, S. Büttcher. Reciprocal Rank Fusion Outperforms Condorcet and Individual Rank Learning Methods. *Proceedings of the 32nd ACM SIGIR Conference*, 2009.

[17] K. Yang. LeanDojo Benchmark 4, phiên bản v10. Zenodo, 2024. `https://zenodo.org/records/12740403`.

[18] mathlib4. `https://github.com/leanprover-community/mathlib4`. Truy cập tháng 10/2026.

[19] B. Efron, R. J. Tibshirani. *An Introduction to the Bootstrap*. Chapman & Hall, 1993.

[20] P. Good. *Permutation, Parametric, and Bootstrap Tests of Hypotheses*, ấn bản 3. Springer, 2005.

[21] B. Phipson, G. K. Smyth. Permutation P-values Should Never Be Zero: Calculating Exact P-values When Permutations Are Randomly Drawn. *Statistical Applications in Genetics and Molecular Biology* 9(1), 2010.

[22] S. Holm. A Simple Sequentially Rejective Multiple Test Procedure. *Scandinavian Journal of Statistics* 6(2), 1979.

[23] A. Z. Broder, D. Carmel, M. Herscovici, A. Soffer, J. Zien. Efficient Query Evaluation Using a Two-Level Retrieval Process. *Proceedings of the 12th ACM CIKM*, 2003.

PHỤ LỤC A

Khai báo sử dụng AI và đóng góp

A.1 Khai báo sử dụng AI

Theo yêu cầu của học phần, nhóm khai báo việc dùng công cụ AI. Nhóm dùng Claude (Anthropic) qua công cụ Claude Code cho các việc sau: lập kế hoạch 10 tuần, viết mã nguồn của pipeline, unit test, các script thí nghiệm và vẽ biểu đồ, bản nháp tài liệu trong thư mục `docs`, và soạn bản báo cáo này. Mọi bảng số liệu trong báo cáo được sinh bằng script từ tệp kết quả, không chép tay từ đầu ra của AI. Ba bản nhận xét phản biện báo cáo (`report/danh_gia_report.md`, `danh_gia_report_lan_2.md` và `danh_gia_report_lan_3.md`) được tạo với Codex (OpenAI); Codex đọc báo cáo, mã nguồn và kết quả rồi chỉ ra các điểm cần sửa, trong đó có lỗi tính p hai phía của hàm bootstrap, các float đặt sai chỗ và cách dùng thuật ngữ. Các thay đổi theo nhận xét, gồm đối chứng V4-K240, tập holdout, kiểm định hoán vị, ghim nguồn của bảng và hình, và phần viết lại báo cáo, được thực hiện với Claude Code. [Nhóm xác nhận Codex chỉ được dùng cho ba bản phản biện này.]

Các bước kiểm chứng tự động đã thực hiện gồm 58 unit test (trong đó có các ví dụ tính tay của giáo trình về mã VB, mã gamma và tần suất tài liệu, ví dụ tính tay cho bootstrap theo cụm, kiểm định hoán vị theo cụm và hiệu chỉnh Holm), so sánh chỉ mục với một cài đặt tham chiếu đơn giản, so mã băm giữa bản xây chỉ mục tuần tự và bản Spark, kiểm tra tính tất định của kết quả và kiểm tra tự động 20 truy vấn mẫu. Những việc AI không quyết định thay nhóm gồm phân vai, nội dung thống nhất với Nhóm 4, đối chiếu thủ công 20 truy vấn mẫu, nhãn thủ công của các trường hợp thất bại và kết luận cuối cùng khi bảo vệ. Mỗi thành viên phải đọc, chạy lại và giải thích được phần mã thuộc vai trò của mình theo hướng dẫn trong `docs/role_guides.md`.

A.2 Phân công và đóng góp

Mỗi vai trò có một người chính và một người dự phòng có thể chạy lại và giải thích phần việc khi bảo vệ (Bảng A.1). Nhật ký đóng góp chi tiết nằm trong tệp `contributions.md`. Các ô màu đỏ là phần nhóm chưa chốt.

Bảng A.1. Vai trò, người phụ trách và phần mã tương ứng

Vai trò	Người chính	Dự phòng	Phần phụ trách
Dữ liệu và trích xuất	Nguyễn Đức Hoàng	Nguyễn Ngọc Minh	tải dữ liệu, trích xuất, tập con, data card, data dictionary, 20 truy vấn kiểm tra
Xây chỉ mục	Trần Duy Khánh	[người phụ trách phục vụ]	xây chỉ mục tuần tự và Spark, nén posting, thí nghiệm xây chỉ mục
Baseline BM25	Nguyễn Mạnh Cường	[người phụ trách đặc trưng]	bộ tách từ, BM25 và BM25F, tinh tham số, đối chiếu Lucene
Đặc trưng cấu trúc	[Vũ Trung Kiên hoặc Trần Đình Dũng]	Nguyễn Mạnh Cường	đặc trưng, bộ lọc accessible premise, thống kê phụ thuộc, reranker, V5
Phục vụ và hiệu năng	[Vũ Trung Kiên hoặc Trần Đình Dũng]	Trần Duy Khánh	API, CLI, kiểm tra tải, profile độ trễ
Đánh giá và phân tích lỗi	Nguyễn Ngọc Minh	Nguyễn Đức Hoàng	chỉ số, kiểm định, ablation, phân tích lỗi, holdout, so sánh với Nhóm 4, báo cáo

PHỤ LỤC B

Tái lập kết quả

B.1 Lệnh tái lập

Các lệnh dưới đây chạy trong thư mục gốc của kho mã với môi trường cài theo `README.md` từ các tệp khóa phiên bản. Có hai mục đích khác nhau. Để dựng lại đúng bản báo cáo đã phát hành, chạy `bash report/latex/build.sh`: lệnh này không chạy lại thí nghiệm, chỉ đọc các lần chạy đã ghim trong `results/figures/report_manifest.json` và `report_records.json`. Trước khi sinh lại bảng và hình, nó so SHA-256 của mọi tệp đầu vào với giá trị ghi trong `tables/SOURCES.md` và `figures/SOURCES.md`, và dừng nếu có tệp lệch; tùy chọn `--compile-only` chỉ biên dịch từ bảng và hình đã có. Để tái tạo kết quả từ đầu hoặc cập nhật báo cáo bằng lần chạy mới, chạy `scripts/reproduce.sh`; script này gọi `pin_runs.py`, tức ghim lại các lần chạy mới nhất, rồi dựng báo cáo bằng `build.sh --update`, nên bảng có thể thay đổi. Tùy chọn `--with-tuning` chạy lại cả quá trình chọn tham số, `--with-scale` chạy các thí nghiệm quy mô, tải và Lucene, `--report` sinh bảng, hình và biên dịch báo cáo.

```
# dựng lại đúng bản đã phát hành từ các lần chạy đã ghim
python -m pytest -q                                          # 58 unit test
bash report/latex/build.sh                  # hoặc --compile-only: chỉ biên dịch

# tái tạo từ đầu, ghim lại lần chạy mới nhất rồi dựng báo cáo
bash scripts/reproduce.sh [--with-tuning] [--with-scale] [--report]

# một biến thể trên một tập, và truy vấn thử
python -m n3.eval.run_eval --config configs/v4_rerank_dep.yaml \
    --eval random_test_all
python -m n3.serving.cli --query-id random:val:36:0 --mode V5 -k 5
uvicorn n3.serving.api:app --port 8000 --workers 4

# cập nhật phân tích sau khi có lần chạy mới (thay đổi bản đã ghim)
python scripts/pin_runs.py && python scripts/ablation_significance.py
python scripts/robustness.py && python scripts/candidate_analysis.py
bash report/latex/build.sh --update

# xây chỉ mục trên Linux, cùng máy
docker build -t n3-spark-linux -f docker/spark-linux.Dockerfile docker
docker run --rm -v <repo>:/repo n3-spark-linux \
    bash /repo/docker/run_build2_linux.sh --reps 3 --sizes 10k,full,x5,x20
```

Bảng nào được sinh từ tệp kết quả nào, các `run_id` đã ghim và mã SHA-256 của mọi tệp đầu vào tại thời điểm sinh bảng được liệt kê trong `report/latex/tables/SOURCES.md`; thứ tự thời gian của các lần chạy nằm trong `docs/process_log.md`.

B.2 Ví dụ một bản ghi lần chạy

Bản ghi dưới đây của lần đánh giá V5 trên tập holdout được rút gọn; bản đầy đủ gồm cả cấu hình, thông tin chỉ mục và mọi chỉ số nằm trong `results/raw/eval`.


```
{"run_id": "eval-V5-20261018-230340-4bf3", "kind": "eval",
 "timestamp": "2026-10-18T16:04:39+00:00", "git_commit": "uncommitted",
 "code_hash": "50e9470e64ad", "data_version": "leandojo_benchmark_4@v10#md5:25e1ee60",
 "parameters": {"config": {"name": "V5"}, "eval_set": "random_holdoutB"},
```

```
  "seed": 42, "status": "ok",
  "metrics": {"R@10": 0.3323, "MRR": 0.2609, "n_queries": 2500}}
```

PHỤ LỤC C

Bảng bổ sung

Các bảng dưới đây bổ sung cho phần thân và được nhắc tới trong các chương tương ứng, xếp theo ba nhóm.

C.1 Dữ liệu và môi trường

Bảng C.1. Nguồn dữ liệu

Thuộc tính	Giá trị
Bộ dữ liệu	LeanDojo Benchmark 4, phiên bản v10, công bố ngày 14/7/2024
Nơi tải	Zenodo record 12740403, tệp `leandojo_benchmark_4.tar.gz`, md5 `25e1ee60cd8925b9d2e8673ddcc34b4c`
Phiên bản thư viện	mathlib4 commit `29dcec07`, trích bằng LeanDojo 2.0.0 [18]
Giấy phép	Bộ dữ liệu: CC BY 2.0. Mã nguồn Lean của mathlib4, Lean 4 và Batteries: Apache 2.0
Nội dung	`corpus.jsonl` (khai báo theo tệp), các tệp `train/val/test.json` của hai split

Bảng C.2. Các bảng sau trích xuất. Bảng `queries` chứa truy vấn của cả hai split, nên một tactic thuộc cả hai split được đếm hai lần

Bảng	Số dòng	Khóa	Cột chính
`premises`	180.957	`premise_id`	tên đầy đủ, tệp, namespace, loại khai báo, phát biểu, mã nguồn, dòng bắt đầu và kết thúc
`queries`	315.014	`query_id`	split, theorem đang chứng minh, tệp, dòng bắt đầu, proof state
`qrels`	674.628	(`query_id`, `premise_id`)	nhãn liên quan nhị phân
`files`	5.674	`path`	danh sách tệp được import trực tiếp
`dead_letter`	136.296	`key`	loại bản ghi bị loại và lý do

Bảng C.3. Môi trường thí nghiệm

Thành phần	Cấu hình
CPU	Intel Core thế hệ 12 (Alder Lake), 6 nhân vật lý, 12 luồng
Bộ nhớ, hệ điều hành	32 GB RAM; Windows 11 (bản 26300)
Python chính	Python 3.13.0; numpy 2.5.4; pyarrow 26.0.0; pandas 3.0.6; scikit-learn 1.9.1; FastAPI 0.143.3; uvicorn 0.54.0
Spark	PySpark 4.0.1 chạy chế độ local với Python 3.11.17 và Java 21.0.6 (PySpark trên Windows không chạy được với Python 3.13)
Linux	container Docker trên cùng máy (máy ảo WSL2 được cấp 12 luồng và 16 GB RAM), Debian 12, Python 3.11.17, Java 17, cùng phiên bản PySpark, numpy, pyarrow và pandas (`docker/spark-linux.Dockerfile`)
Lucene	Anserini 2.3.0 (Lucene 10.5) trên Java 21.0.6
Lặp và seed	seed 42 cho mọi lấy mẫu; thí nghiệm thời gian lặp 3 lần, báo trung vị cùng giá trị nhỏ nhất và lớn nhất
Giới hạn thời gian	mỗi yêu cầu của client kiểm tra tải có timeout 120 giây

C.2 Truy hồi, phục vụ và xây chỉ mục

Bảng C.4. Số liệu công bố trong bài báo LeanDojo đặt cạnh kết quả của nhóm, chỉ để làm bối cảnh: các dòng của bài báo đo trên benchmark và snapshot khác nên không so sánh trực tiếp được

	`random`			`novel_premises`		
Hệ thống	R@1	R@10	MRR	R@1	R@10	MRR
BM25, toàn bộ premise (LeanDojo, Bảng 1, Lean 3)	0,019	0,119	0,08	0,021	0,124	0,08
B0 của nhóm	0,018	0,100	0,065	0,024	0,127	0,090
BM25, accessible premise (LeanDojo, Bảng 1, Lean 3)	0,067	0,172	0,15	0,059	0,155	0,14
V2a của nhóm	0,053	0,183	0,148	0,062	0,221	0,176
V4 của nhóm	0,072	0,243	0,196	0,072	0,269	0,211
V5 của nhóm	0,073	0,281	0,217	0,069	0,280	0,215
ReProver (LeanDojo, Bảng B, Lean 4, snapshot cũ hơn v10)	0,128	0,347	0,29	0,098	0,321	0,24

Bảng C.5. Tinh chỉnh số candidate bổ sung của V5 trên validation `random` (2.536 truy vấn). V4 trên cùng tập: Recall@10 = 0,273; Recall@50 = 0,398; MRR = 0,220

Tệp	Thư mục	Phổ biến	Số candidate	Candidate recall	R@10	R@50	MRR
0	0	0	100	0,420	0,279	0,398	0,225
50	0	0	121	0,509	0,288	0,462	0,226
0	50	0	139	0,525	0,288	0,467	0,228
0	0	50	150	0,517	0,292	0,455	0,228
50	50	0	154	0,555	0,288	0,487	0,228
50	50	50	204	0,615	0,299	0,501	0,229
100	**100**	**50**	**232**	**0,634**	**0,300**	**0,505**	**0,232**
100	100	100	282	0,659	0,295	0,510	0,231

Bảng C.6. BM25 của Lucene 10.5 (Anserini 2.3.0) so với B0 trên tập test `random`

Hệ thống	R@10	R@50	MRR	Trùng top 10	ms/truy vấn
Lucene, bộ phân tích tiếng Anh, $k_1 = 0{,}9$, $b = 0{,}4$	0,0757	0,1386	0,0508	0,544	2,3
Lucene, bộ phân tích tiếng Anh, $k_1 = 1{,}2$, $b = 0{,}75$	0,0889	0,1551	0,0580	0,594	2,3
Lucene, token của B0, $k_1 = 1{,}2$, $b = 0{,}75$	0,1003	0,1760	0,0657	0,989	2,4
B0 của nhóm, $k_1 = 1{,}2$, $b = 0{,}75$	0,1004	0,1763	0,0654	1	13,2

Ghi chú: trường statement, một thread. Trùng top 10 là tỉ lệ phần tử chung giữa top 10 của Lucene và của B0, query-macro; thời gian là trung bình ở trạng thái ổn định (bỏ 100 truy vấn đầu), không gồm khởi động JVM.

Bảng C.7. Thời gian theo bước của một truy vấn (ms, trung vị và P95 trong ngoặc), một tiến trình, chỉ mục đầy đủ, 500 truy vấn, bộ đệm đã nóng

Bước	B0	V2	V4
Tokenize	0,03 (0,08)	0,18 (0,56)	0,17 (0,53)
Cộng điểm BM25/BM25F	12,41 (23,54)	19,71 (36,10)	18,26 (32,24)
Tính mặt nạ accessible premise			0,85 (1,24)
Lọc và chọn top 100	0,67 (1,09)	0,76 (1,14)	0,87 (1,42)
Tính 13 đặc trưng			0,54 (0,96)
Rerank và sắp xếp			0,06 (0,10)
Tổng	13,14 (24,72)	20,66 (37,59)	20,89 (35,69)
Tỉ lệ thời gian của BM25	95%	96%	88%

Bảng C.8. Kiểm tra tải trên chỉ mục đầy đủ theo số worker (P95 tính bằng ms và QPS, trung vị của 3 lần; RAM là tổng của mọi tiến trình server)

			10 client		50 client		100 client	
Biến thể	Worker	RAM (GB)	P95	QPS	P95	QPS	P95	QPS
B0	4	3,4	62	250	334	242	572	240
B0	8	6,7	58	229	365	253	865	235
B0	12	9,8	57	260	376	260	748	260
V4	4	3,4	105	136	541	143	1.077	142
V4	8	6,7	92	159	532	162	997	161
V4	12	9,8	103	146	584	162	1.185	148

Bảng C.9. So sánh hai engine Spark, chỉ mục B0, Spark local[4] (trung vị của 3 lần; engine RDD ở 3,6 triệu bản ghi chạy 1 lần trên mỗi nền tảng; engine RDD ở 905K chỉ đo trên Windows)

		Thời gian (s)		RAM đỉnh (GB)	
Kích thước	Engine	Windows	Linux	Windows	Linux
905K	RDD	41,0		3,8	
905K	Arrow	23,5	19,2	1,5	2,8
3,6 triệu	RDD	106,7	109,7	11,4	12,2
3,6 triệu	Arrow	35,7	35,7	2,0	3,9

C.3 Thiết kế thí nghiệm

Bảng C.10. Thiết kế thí nghiệm và số lần chạy, đếm từ các bản ghi trong `results/raw`. Một lần đo tải là một mức client của một cấu hình trong một lần lặp, gồm 400 yêu cầu được tính

Thí nghiệm	Các mức	Số lần chạy
Chất lượng	8 biến thể; validation và kiểm tra của `random`, kiểm tra của `novel_premises`	24
Đối chứng và holdout	V4-K240 trên 4 tập, V2-nons trên 3 tập, V4-poponly và V4-pronly trên validation và kiểm tra `random`; B0, V2, V4, V5 trên holdout	15
Chất lượng theo quy mô	B0, V2, V4; chỉ mục 10K, 50K, 100K, 181K; mẫu 500 truy vấn	12
Xây chỉ mục, engine RDD	chỉ mục B0; tuần tự, Spark local[2], local[4], local[8]; 10K, 50K, 100K, 181K, 905K; Windows; 3 lần	60
Ablation engine RDD	không kết hợp cục bộ và tách khóa nóng 4 phần (181K, 905K, local[4]); codec raw32 và gamma (181K, tuần tự); 3 lần	18
Xây chỉ mục, engine Arrow	chỉ mục B0 và ba trường; tuần tự, Spark local[2], local[4], local[8]; 10K, 181K, 905K, 3,6 triệu; Windows và Linux; 3 lần	192
Engine RDD ở 3,6 triệu	chỉ mục B0, Spark local[4]; Windows và Linux	2
Tải	B0, V2, V4; 4 kích thước chỉ mục; 1, 10, 50 client; 3 lần (36 cấu hình, 43.200 yêu cầu)	108 lần đo
Tải theo số worker	B0, V2, V4; 4, 8, 12 worker; 1, 10, 50, 100 client; chỉ mục đầy đủ; 3 lần	108 lần đo
Kiểm tra CPU khi chịu tải	B0; 4, 8, 12 worker; một và bốn tiến trình client	6 lần đo
Tải V4, V4-K240, V5	cùng server 4 worker; 1, 10, 50 client; 3 lần, thứ tự xoay vòng (thêm 18 lần đo V4, V5 ở phiên trước)	27 lần đo
Ablation reranker	V4 đầy đủ và 9 biến thể, mỗi biến thể huấn luyện lại; kiểm tra `random`	10
Cách mã khi truy vấn	raw32, VB, gamma; mẫu 500 truy vấn; tắt bộ đệm 3 lần, bật bộ đệm 1 lần	12 lần đo
Profile độ trễ	B0, V2, V4; 500 truy vấn; lượt lạnh và lượt nóng	1
Tinh chỉnh V5	8 tổ hợp nguồn ứng viên; validation `random`; huấn luyện lại mỗi tổ hợp	8
BM25 của Lucene	Anserini 2.3.0, ba biến thể; B0 đo một thread cùng quy tắc; kiểm tra `random`	4